WEEK20:

1. We write the declarative code (SQL or Python)
2. We create a DLT pipeline to run the code.

DEMO - 1:

Orders → Transactional Data
Customers data → Master Data

Medallion Architecture:

Landing → Bronze (RAW)→ Silver → Gold

In landing zone:
==========
Azure storage account:
Trendytech-dlt-demo-sa
-landing
- orders (orders1.csv)
- customers (customers1.csv)

In landing we will have raw data in csv format.

Bronze → we will have data in delta format (tables created)
Bronze will act like the source of truth.

Useful for: Data engineers

Silver → (cleaning,enrich,Merge,SCD2)

- Silver cleaned (implementing data quality checks)
- Silver merged (use SCD2 or simple merge command)

For orders is more of my fact data (I do not maintain the history like from chaning orders status from pending to shipped etc)

Customers (is more like my dimesion I will implement SCD2 → SO I will maintain history as well)

Useful for : Machine learning team / Data scientists / Data analysts

Gold → Domain specific data (Marketing / sales) aggregate for reporting.

Useful for: Business users

Writing Declarative ETL Code in Delta Live Tables

In landing zone:
==========
Azure storage account:
Trendytech-dlt-demo-sa
-landing
- orders (orders1.csv)
- customers (customers1.csv)

Step1: Load the data from landing zone and move it to bronze layer.

```
%sql
CREATE STREAMING LIVE  TABLE orders_bronze
AS
SELECT *,
_metadata.file_name as filename,
Current_timestamp() as load_time
FROM cloud_files ('abfss://retail@dltdemo112.dfs.core.windows.net/input/orders',
'csv', map("cloudFiles.inferColumnTypes", "True"));

CREATE STREAMING LIVE  TABLE customers_bronze
AS
SELECT *,
_metadata.file_name as filename,
```

```
Current_timestamp() as load_time
FROM cloud_files ('abfss://retail@dltdemo112.dfs.core.windows.net/input/customers',
'csv', map('cloudFiles.inferColumnTypes', 'True'));
```

Any Delta live table is a delta table under the hood

==========

Step2: Take the data from bronze, clean it and load it to silver cleaned.

```
%sql
CREATE STREAMING TABLE orders_silver_cleaned
(
CONSTRAINT valid_orders EXPECT (order_id is not null) ON VIOLATION DROP ROW,
CONSTRAINT valid_customer EXPECT (customer_id is not null) ON VIOLATION DROP ROW
) as
SELECT  orderid as order_id,
      orderdate as order_date,
      customerid as customer_id,
      Filename as file_name,
      totalamount as total_amount,
      status,
    load_time
FROM stream(live.orders_bronze);

CREATE STREAMING TABLE customers_silver_cleaned
(
CONSTRAINT valid_customer EXPECT (customer_id is not null) ON VIOLATION DROP ROW
) as
SELECT  customerid as customer_id,
    customername as customer_name,
    address as city,
    dateofbirth as dob,
    registrationdate as customer_since,
    Filename as file_name,
    load_time
```

```
FROM stream(live.customers_bronze);
```

Step3: customers → Take the data from silver cleaned, handle duplicates (SCD2) and load in silver layer.

```
%sql
CREATE STREAMING TABLE customers_silver;

CREATE FLOW customers_silver_flow AS AUTO CDC
into live.customers_silver FROM STREAM(live.customers_silver_cleaned)
Keys (customer_id)
Sequence by load_time
Stored as scd type 2;
```

Step4: Orders → Take the data from silver cleaned, handle duplicates (merge) and load in silver layer.

```
%sql
CREATE STREAMING TABLE orders_silver;

APPLY CHANGES into live.orders_silver FROM STREAM(live.orders_silver_cleaned)
Keys (order_id)
Sequence by load_time;
```

It will DO the merge statement.

Step5: Take the data from silver and perform aggregations and load to gold layer.

```
%sql
CREATE  MATERIALIZED VIEW city_wise_sales_gold
As
SELECT c.city,sum(total_amount) as total_sales
From live.orders_silver o
JOIN live.customers_silver c
On o.customer_id=c.customer_id
Group by c.city;
```

DLT has 2 core abstractions:

- **Streaming table** (it is a delta table) it as a stream writing to it. It is used for ingesting huge amounts of data where we can't afford to do recomputation.
- **Materialized view**: The results of a query stored in a delta table. It is the answer to the query.

Expectations:

- Do not drop just a raise a warning
- Drop bad records
- Abort processing for a single bad record.

- ➢ For the incremental processing we can use streaming live table (older) – streaming table (new term).
- ➢ Live table (older DLT term) → materialized view (new DLT term) → for full load processing.

For materialize view (Enzyme Folder): we can view in azure storage account.

Enzyme → set of optimization to make sure.....

External Locations and Modern Implementation using unity catalog

- ✓ We need to change from Hive metastore to unity catalog.
- ✓ From Mount point to external location.
- ✓ We will be doing some code changes based on latest and updated style.
- ✓ There is no need for use of live keyword.

Guidelines for Next Demo Set:

Hive metastore → Unity catalog

Classic compute → serverless

Mount point → Volumes / External location

Live Keyword → remove that and use 3 level namespace.

DEMO - 2:

We will download a public path and copy in volume.

Catalog

- Schema
  - Tables
  - Views
  - Volumes (files)

We will create a volume.

Inside this created volume we will copy a file from internet.

```
%sql
use catalog trendytech_dev;
```

```
my_catalog="trendytech_dev"
my_schema="demo3_db"
my_volume="demo3_volume"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {my_catalog}.{my_schema}")
spark.sql(f"CREATE VOLUME IF NOT EXISTS {my_catalog}.{my_schema}.{my_volume}")

volume_path=f"/Volumes/{my_catalog}/{my_schema}/{my_volume}/"
download_url="https://health.data.ny.gov/api/views/jxy9-yhdk/rows.csv"
filename="babynames.csv"

dbutils.fs.cp(download_url,volume_path +filename)
```

This error happens because **spark.sql() cannot be used to run DDL or DML commands (like ALTER, DROP, CREATE, or INSERT) inside Spark Declarative Pipelines (SDP)**

```
%sql
CREATE OR REFRESH STREAMING TABLE baby_names_raw
COMMENT "popular baby names in NEW york"
AS SELECT Year,`First Name` as first_name,county,Sex,Count
FROM STREAM(read_files
        ("/Volumes/{my_catalog}/{my_schema}/{my_volume}/",
        format="csv",
        header=true,
        mode=> 'FAILFAST'
        ))
```

```
%sql
CREATE OR REFRESH MATERIALIZED VIEW baby_names_prepared (
    CONSTRAINT valid_first_name EXPECT (first_name is NOT NULL),
    CONSTRAINT valid_first_name EXPECT (first_name is NOT NULL)
)
COMMENT 'baby first name data cleanded and prepare for analysis'
AS
SELECT
    Year As year_of_birth,
    First_Name,
    count
FROM LIVE.baby_names_raw;
```

```
%sql
CREATE OR REFRESH MATERIALIZED VIEW top_baby_names_2021
COMMENT 'A table summarizing counts of the top baby names for NEw york for 2021'
AS
SELECT
    First_Name,
    sum(count) as total_count
FROM LIVE.baby_names_prepared
```

```
where year_of_birth=2021
Group by First_Name
ORDER BY total_count desc
limit 10;
```

DEMO - 3:

We can select more than one notebook as the code....

- Removed the mount point
- Converted from hive metastore to unity catalog
- Live keyword no longer required
- CREATE FLOW orders_silver_flow as AUTO CDC (*instead of apply changes*)
- Serverless

ETL pipeline 1:

Whatever the name you give to the pipeline is the root foler.

Sales_pipeline_1:

- Explorations
- Transformations

- explorations: Ad-hoc notebooks used to explore the data processed by this pipeline.

- transformations: All dataset definitions and transformations.

- utilities: Utility functions and Python modules used in this pipeline.

1. **Use STREAM as a prefix** (e.g., FROM STREAM read_files(...)) when ingesting raw objects via file readers.
2. **Use STREAM() with parentheses** (e.g., FROM STREAM(my_catalog.my_schema.my_table)) *only* when the source is an existing, already-defined Delta table inside your catalog

ETL PIPELINE-1:

```
-- Reading file from raw and form a table bronze table

CREATE OR REFRESH STREAMING TABLE customers_bronze
AS
select *,
_metadata.file_name as source_file_name,
current_timestamp() as load_time
from stream read_files(
    '/Volumes/dlt_flow/default/raw_data/customers/',
    format=> 'csv',
    inferSchema=> 'true'
);

CREATE OR REFRESH STREAMING TABLE orders_bronze
AS
select *,
_metadata.file_name as source_file_name,
current_timestamp() as load_time
from stream read_files(
    '/Volumes/dlt_flow/default/raw_data/orders/',
    format=> 'csv',
    inferSchema=> 'true'
);




-- STEP 2:
CREATE STREAMING TABLE orders_silver_cleaned
(
CONSTRAINT valid_orders EXPECT (order_id is not null) ON VIOLATION DROP ROW,
CONSTRAINT valid_customer EXPECT (customer_id is not null) ON VIOLATION DROP ROW
```

```
) as
SELECT  orderid as order_id,
      orderdate as order_date,
      customerid as customer_id,
      source_file_name as file_name,
      totalamount as total_amount,
      status,
    load_time
FROM stream(orders_bronze);

CREATE STREAMING TABLE customers_silver_cleaned
(
CONSTRAINT valid_customer EXPECT (customer_id is not null) ON VIOLATION DROP ROW
) as
SELECT  customerid as customer_id,
    customername as customer_name,
    address as city,
    dateofbirth as dob,
    registrationdate as customer_since,
    source_file_name as file_name,
    load_time
FROM stream(customers_bronze);


-- step 3
CREATE STREAMING TABLE customers_silver;

CREATE FLOW customers_silver_flow AS AUTO CDC
into customers_silver FROM STREAM(customers_silver_cleaned)
Keys (customer_id)
Sequence by load_time
Stored as scd type 2;



-- step 4
CREATE STREAMING TABLE orders_silver;

CREATE FLOW orders_silver_flow AS AUTO CDC
INTO orders_silver FROM STREAM(orders_silver_cleaned)
Keys (order_id)
Sequence by load_time;
```

```sql
--step5:
CREATE  MATERIALIZED VIEW city_wise_sales_gold
As
SELECT c.city,sum(total_amount) as total_sales
From orders_silver o
JOIN customers_silver c
On o.customer_id=c.customer_id
Group by c.city;
```

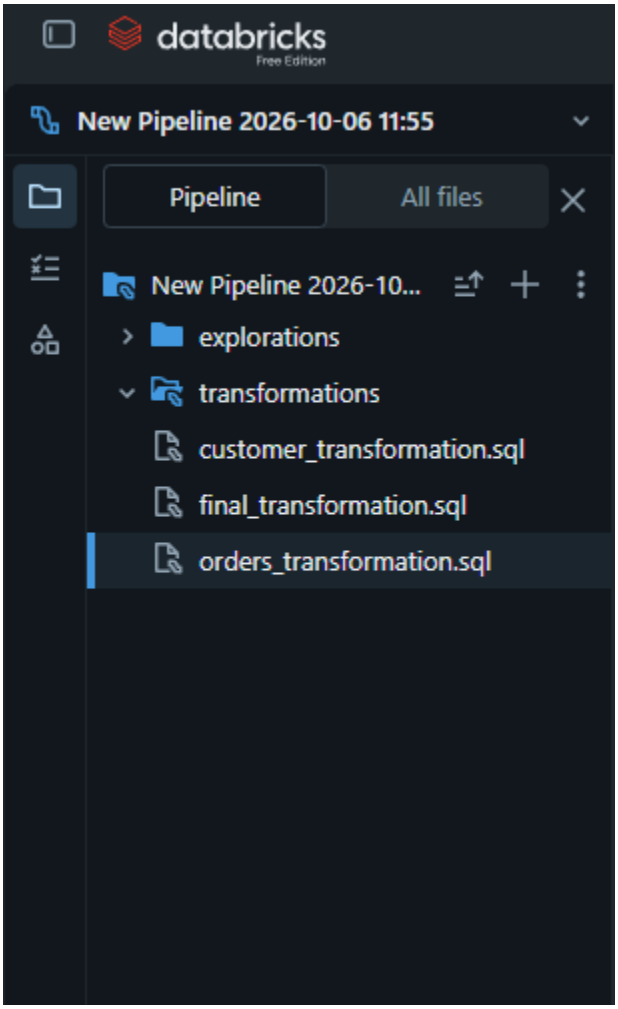
Etl pipeline 2:

Common pipeline settings:

- Compute related (serverless, compute)
- Code assets
- Configurations (parameters)

  Key : source

  Value : /volumes/.....



You already processed 100 Files which are in cloud storage. By mistake you clicked on **Full refresh...** may be those 100 files are deleted due to retention policy.. in that case we lost our data.

Truncated your delta tables (bronze,silver,gold)

This can leads to data loss. So be careful when you run full refresh....

The table propery to avoid full refresh:

Pipelines.reset.allowed=false

The full refresh will not allowed to trigger..... materialized views that use expectations are always fully refreshed...

PERSONAL LEARNING:

Lakeflow Declarative Pipelines:

We understood lakeflow connect for data ingestion....

Once the data is ingested in Delta lake, next step is to perform the transformation (ETL activity)

Databricks has made it super easy for data engineers by introducing Lakeflow Declarative Pipelines (earlier calles as Delta Live Tables)

So Basically they recently rebranded DLT as lakeflow declarative pipelines. Most of things are still the same. Its just that they give a new IDE for the development which makes life of developer easy. There are slight changes.

As per me, its important to know DLT before this rebranding happened. WHY ?

Because the most of the development work that would have been done in your company would be using the previous IDE.

Its important to know the new changes like the new IDE for development.

So the next 3 videos will be based on the DLT (before it was rebranded to declarative pipeline). These sessions will be really really valuable for as you in the companies if you see exisiting pipelines, you would see it this way).

I will like to highlight a few things :

- Databricks open sourced its declarative pipeline framework, and contibuted it to the apache spark open source project.
- The databricks workspace creation, you remember the option in networking so that we save cost.
- Azure Data factory would be used for ingesting the data to delta lake.
- Created mount point to refer to the container that we created in storage account.

You just declare what you want – and DLT takes care of how it should be done...

QUALITY CHECKS:

Built-in DDLT supports something called expectations – where you define rules (like "ths column should never be null"). If a record breaks the rule, DLT will **drop** it or send it to **quarantine**- and you don't have to write extra code to handle it.

Automatic Dependency management:

You don't need to worry about the order of transformations. Just define your tables and DLT figures out which table depends on which and runs things in the correct order.

INCREMENTAL PROCESSING:

DLT is smart – it only process new or changed data using something called change Data capture (CDC).

UNIFIED BATCH AND STREAMING:

Normally, Batch and streaming pipelines are built and maintained separately. But with DLT you can use the same code for both. It detects whether your data source is streaming and handles it accordingly..

Building Declarative ETL Pipelines with Delta Live Tables (DLT)

What is DLT ?

- DLT is A framework to build ETL pipelines with a Declarative approach.

What is Declarative Approach ?

- We tell what to do (not how we do it)

We can Say simplify traditional ETL.

Traditional ETL and its challenges:

- How to extract the data
- How to transform the data
- How to load it
- We look for Orchestration framework to stitch things together.
- Infra management, Scaling up and scaling down the infrastructure.
- Manage depedecies
- Track lineage
- Handle batch and streming in the pipeline
- Ensure data quality and handle failure and retries
- Monitor and optimize pipeline
- Deploy in multipe Environment.

Time spent on tooling kind of dominate your efforts.

DLT solves the above issues, and make it easy to build ETL pipelines.

Why Declarative programming ?

Declarative programming is highly favored in Distributed Ledger Technology (DLT) and blockchain development because it prioritizes safety, predictability, and formal verification.

Understanding your objective allows DLT to take care of all the boring stuff.

1. STREAMING TABLE:

- A **streaming table** processes data incrementally as it arrives. Instead of repeatedly scanning the entire source, it tracks what has already been processed and consumes new data during each pipeline update.
- Good for append-only sources (logs, events, Kafka, files landing in a folder)
- Fast and cheap, because it never reprocesses old data

Example

Imagine an orders application continuously produces JSON files:

- orders_001.json
- orders_002.json
- orders_003.json

A streaming table can process each new file incrementally and append the records to a Bronze table. It does not need to reread every historical file on every run.

```
%sql
CREATE OR REFRESH STREAMING TABLE bronze_orders
COMMENT 'Raw order files ingested incrementally'
AS
SELECT
    *,
    _metadata.file_path AS source_file,
    _metadata.file_modification_time AS source_file_modified_time,
    current_timestamp() AS ingestion_timestamp
FROM STREAM read_files(
    '/Volumes/landing/orders',
    format => 'json',
    schemaEvolutionMode => 'addNewColumns'
);
```

schemaEvolutionMode options

Mode	Behavior	Recommended use
addNewColumns	Adds newly discovered columns. The stream may need a restart when a new column is detected.	Controlled source changes
rescue	Keeps the main schema stable and places unexpected fields or type mismatches into _rescued_data.	Untrusted or frequently changing sources
failOnNewColumns	Stops when a new column appears.	Strict data contracts
none	Does not automatically evolve the schema. New fields may be ignored, dropped, or rescued depending on configuration.	Fixed schemas
addNewColumnsWithTypeWidening	Adds new columns and allows compatible type widening, such as integer to long.	Sources with controlled type expansion

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
COMMENT 'Incrementally ingested raw order files'

AS

SELECT
    *,
    _metadata.file_path AS source_file_path,
    _metadata.file_name AS source_file_name,
    _metadata.file_size AS source_file_size,
    _metadata.file_modification_time AS source_file_modification_time,
    current_timestamp() AS ingestion_timestamp
```

```
FROM STREAM read_files(
    '/Volumes/<catalog>/<schema>/<volume>/orders',

    -- Required: source file format
    format => 'json',

    -- Schema handling
    schemaEvolutionMode => 'addNewColumns',
    rescuedDataColumn => '_rescued_data',


    -- Optional schema behavior
    inferColumnTypes => true,

    -- JSON-specific options
    multiLine => false,

    -- Optional parsing options
    mode => 'PERMISSIVE',

    -- CSV-specific options; remove for JSON
    header => true,
    delimiter => ',',
    quote => '"',
    escape => '\\',

    -- Optional schema hints
    schemaHints => 'order_id STRING, amount DECIMAL(18,2)',

    -- Optional rescued-data column
    rescuedDataColumn => '_rescued_data'

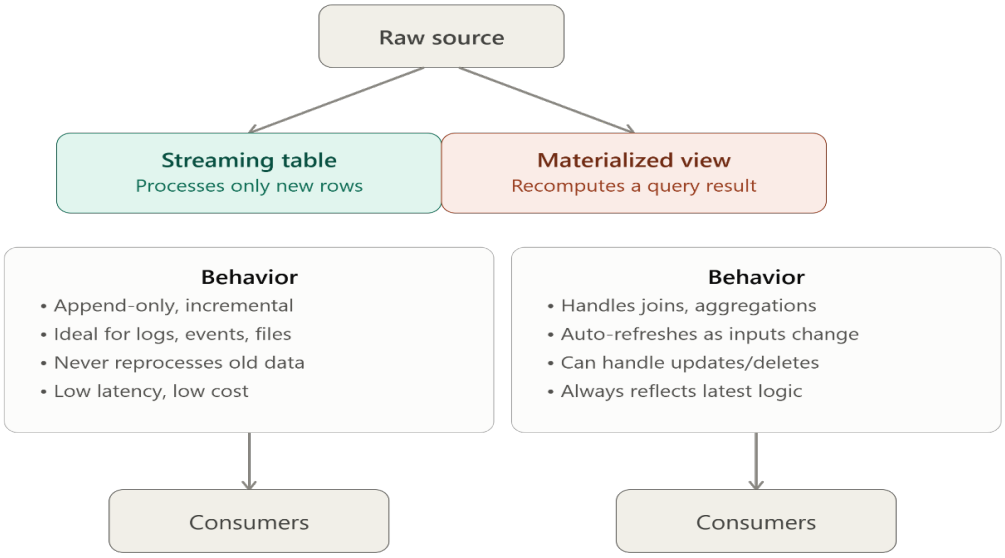
);
```

- Schema hints tell the ingestion process what type selected columns should have.
- STREAM Tells Databricks to read the source incrementally as a streaming relation.
- multiLine => false -→ Set this to true when a single JSON record can span multiple lines.

Materialized view

- Recomputes results from a query, keeping them precomputed and fresh
- Good for aggregations, joins, or dedup logic over the full or changing dataset
- The engine automatically figures out what needs updating
- It stores this prepared result so dashboards do not need to repeatedly join and aggregate the raw tables.
- Materialized views are especially useful for analytics, reporting, and serving optimized datasets

- Receiving new records → **Streaming table**
- Maintaining a business result → **Materialized view**
- Receiving CDC updates/deletes → **Streaming table with AUTO CDC**
- Producing joins, metrics, or dashboards → **Materialized view**



In one line: a streaming table is for "new data keeps arriving, just add it" — a materialized view is for "this result depends on logic that needs recalculating."

Views:

Batch view

- Reads from a batch (static/complete) source
- Recomputed fully, in one shot, whenever it's queried
- Useful for lightweight transformations you don't need to store separately

Streaming view

- Reads from a streaming source (like a streaming table)
- Recomputed incrementally as new data flows through, but still not stored
- Useful as a transformation step between a streaming table and a materialized view, without paying storage cost

```
CREATE OR REPLACE MATERIALIZED VIEW catalog_name.schema_name.view_name
[CLUSTER BY (col_name [, ...])]
[COMMENT 'view_comment']
[SCHEDULE CRON 'cron_string' [AT TIME ZONE 'timezone_id'] | TRIGGER ON UPDATE]
AS
SELECT column1, column2 AS alias_name, ...
FROM base_table
[GROUP BY column1];
```

Inside a pipeline, the keyword uses CREATE OR REFRESH instead of CREATE OR REPLACE

```
CREATE OR REFRESH MATERIALIZED VIEW filtered_taxi_data
COMMENT "Aggregated taxi data inside a pipeline"
AS SELECT
    pickup_zip,
    sum(fare_amount) AS total_fare
FROM LIVE.taxi_raw
GROUP BY pickup_zip;
```

The Core Mechanism: Incremental Refresh

Instead of computing the entire dataset from scratch on every tick, Databricks uses its built-in **Enzyme engine** to automatically perform an **incremental refresh** on the Materialized View inside the continuous pipeline. It looks at the changes in your upstream tables and merges only the differences into the view

2. Merge command

MERGE [WITH SCHEMA EVOLUTION] INTO target_table [AS t]

USING source_table_or_query [AS s]

ON <match condition>

WHEN MATCHED [AND <cond>] THEN UPDATE SET ... | DELETE

WHEN NOT MATCHED [BY TARGET] [AND <cond>] THEN INSERT ...

WHEN NOT MATCHED BY SOURCE [AND <cond>] THEN UPDATE SET ... | DELETE

Clause	Row exists in...	Typical action
WHEN MATCHED	source and target	UPDATE or DELETE
WHEN NOT MATCHED (BY TARGET)	source only	INSERT
WHEN NOT MATCHED BY SOURCE	target only	UPDATE (e.g. soft delete) or DELETE

```
%sql
merge into customer_target as t
using customers_base cb
on t.id=cb.id
when matched then update  set
t.name=cb.name,t.city=cb.city,t.updated_at=cb.updated_at
when not matched then insert (id,name,city,updated_at,op) values
(cb.id,cb.name,cb.city,cb.updated_at,false);
```

Simple insert:

```
MERGE INTO customers t
USING updates s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Insert only (dedupe load, skip existing)

```
MERGE INTO customers t
USING updates s ON t.id = s.id
WHEN NOT MATCHED THEN INSERT *;
```

Update only if something changed

```
MERGE INTO customers t
USING updates s ON t.id = s.id
WHEN MATCHED AND (t.name <> s.name OR t.city <> s.city) THEN
  UPDATE SET t.name = s.name, t.city = s.city, t.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT *;
```

This avoids rewriting unchanged files, which is a big performance win.

Only apply newer records

```
WHEN MATCHED AND s.updated_at > t.updated_at THEN UPDATE SET *
```

Streaming + MERGE PYTHON:

```
def upsert(batch_df, batch_id):
    (DeltaTable.forName(spark, "customers").alias("t")
       .merge(batch_df.alias("s"), "t.id = s.id")
       .whenMatchedUpdateAll()
       .whenNotMatchedInsertAll()
       .execute())

(spark.readStream.table("customers_bronze")
  .writeStream.foreachBatch(upsert)
  .option("checkpointLocation", "/checkpoints/customers")
  .start())
```

3.Upsert command :

There is no separate upsert command we can achive using merge only.

a database operation that inserts a new row if it does not already exist, or updates the existing row if a unique constraint or primary key conflict is triggered.

```
MERGE INTO production.employees AS target
USING staging.employees_updates AS source
ON target.id = source.id
WHEN MATCHED THEN
  UPDATE SET *
WHEN NOT MATCHED THEN
  INSERT *;
```

Databricks handles new incoming columns natively using the WITH SCHEMA EVOLUTION clause to safely adapt your Delta table structure over time.

```
MERGE WITH SCHEMA EVOLUTION INTO production.employees AS target
USING staging.employees_updates AS source
ON target.id = source.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Handling Source Duplicates (Change Data Capture)

Databricks throws a DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE error if your source contains multiple records with the same key. You can resolve this by using a Common Table Expression (CTE) with window functions to deduplicate the source data inline before merging.

[DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE] Cannot perform Merge as multiple source rows matched and attempted to modify the same target row in the Delta table in possibly conflicting ways. By SQL semantics of Merge, when multiple source rows match on the same target row, the result may be ambiguous as it is unclear which source row should be used to update or delete the matching target row. You can preprocess the source table to eliminate the possibility of multiple matches.

Normally, SQL executes the WHERE clause **before** it calculates window functions. Because of this, you cannot filter on a window function right away.

❌ This will throw an error:

```
%sql
SELECT id, name, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) as rn
FROM staging.employees_updates
WHERE rn = 1; -- Error! 'rn' doesn't exist yet when WHERE runs.
```

✅ This works perfectly:

```
%sql
SELECT *
FROM staging.employees_updates
QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1;
```

```
MERGE INTO production.employees AS target
USING (
  SELECT *
  FROM staging.employees_updates
  QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1
) AS source
ON target.id = source.id
WHEN MATCHED THEN
  UPDATE SET *
WHEN NOT MATCHED THEN
  INSERT *;
```

4. SCD TYPE-1 :

Definition

Type 1 SCDs overwrite old data with new data. No historical data is preserved—only the latest value is retained.

Use case example

If a customer's email address changes, and you don't need to track previous addresses, a Type 1 update will replace the old email with the new one.

SCD Type 1 stores only the **latest/current value**.

- Old values are overwritten.
- No historical versions are retained.
- Deleted records are removed.
- Target table represents the current state only.

Key identifies → Sequence orders → Operation acts → Type 1 overwrites

```
CREATE OR REFRESH STREAMING TABLE main.cdc_tutorial.users_current;

CREATE FLOW apply_users_cdc
AS AUTO CDC
INTO main.cdc_tutorial.users_current
FROM stream(main.cdc_tutorial.users_cdf)
KEYS (userId)
IGNORE NULL UPDATES
APPLY AS DELETE WHEN operation = 'DELETE'
APPLY AS TRUNCATE WHEN operation = 'TRUNCATE'
SEQUENCE BY sequenceNum
COLUMNS * EXCEPT (operation, sequenceNum)
STORED AS SCD TYPE 1;
```

- ✓ KEYS (userId) Identifies each user record.
- ✓ IGNORE NULL UPDATES Prevents null update values from overwriting existing values.
- ✓ APPLY AS DELETE WHEN Treats matching events as deletes.
- ✓ APPLY AS TRUNCATE WHEN Clears the entire target for matching events.
- ✓ SEQUENCE BY sequenceNum Determines the logical order of changes.
- ✓ COLUMNS * EXCEPT Writes all source columns except technical CDC columns.

Why SEQUENCE BY matters

Suppose these events arrive physically out of order:

userId = 125, city = Guadalajara, sequenceNum = 6

userId = 125, city = Mexicali, sequenceNum = 5

SEQUENCE BY sequenceNum recognizes that sequence 6 is newer than sequence 5.

Final value: city = Guadalajara

- ❖ **Insert:** create the record.
- ❖ **Update:** overwrite in Type 1; version the record in Type 2.
- ❖ **Delete:** remove in Type 1; close the active version in Type 2.
- ❖ **Truncate:** clear everything in Type 1; unsupported in Type 2.
- ❖ **Sequence:** determines the correct order for both types.
- ❖ **Key:** determines which entity the event affects.

```
from pyspark import pipelines as dp
from pyspark.sql.functions import expr

dp.create_streaming_table(
    "main.cdc_tutorial.users_current"
)
```

```
dp.create_auto_cdc_flow(
    target="main.cdc_tutorial.users_current",
    source="main.cdc_tutorial.users_cdf",
    keys=["userId"],
    ignore_null_updates=True,
    apply_as_deletes=expr("operation = 'DELETE'"),
    apply_as_truncates=expr("operation = 'TRUNCATE'"),
    sequence_by="sequenceNum",
    except_column_list=["operation", "sequenceNum"],
    stored_as_scd_type=1
)
```

Note the Python parameter is apply_as_deletes—plural. Truncates are supported for SCD Type 1

5. SCD TYPE-2 :

customer_id	name	city	operation	updated_at
1	Asha	Pune	INSERT	2026-01-01
2	Ravi	Delhi	INSERT	2026-01-01
1	Asha	Mumbai	UPDATE	2026-03-01
2	Ravi	Delhi	DELETE	2026-04-01

o/p:

customer_id	name	city	updated_at	__START_AT	__END_AT
1	Asha	Pune	2026-01-01	2026-01-01	2026-03-01
1	Asha	Mumbai	2026-03-01	2026-03-01	-
2	Ravi	Delhi	2026-01-01	2026-01-01	2026-04-01

```
CREATE OR REFRESH STREAMING TABLE bronze_customer
AS SELECT * FROM STREAM(advance_topics.scd_type2.customers_cdc);

CREATE OR REFRESH STREAMING TABLE customer_silver;

CREATE FLOW dim_customer_flow as AUTO CDC
INTO customer_silver
from stream(bronze_customer)
keys (customer_id)
ignore null updates
apply as delete when operation='DELETE'
sequence by updated_at
columns * except (operation)
stored as scd type 2;
```

INSERT INTO advance_topics.scd_type2.customers_cdc VALUES

(3, 'Naresh', 'Hyd', 'INSERT', DATE'2026-01-01');

INSERT INTO advance_topics.scd_type2.customers_cdc VALUES

(3, 'Naresh', 'Chenni', 'UPDATE', DATE'2026-01-08');

INSERT INTO advance_topics.scd_type2.customers_cdc VALUES

(3, 'Naresh', 'Banglore', 'UPDATE', DATE'2026-01-10');

O/p:

3	Naresh	Banglore	2026-01-10	2026-01-10	-
3	*Naresh*	*Chenni*	*2026-01-08*	*2026-01-08*	*2026-01-10*
3	*Naresh*	*Hyd*	*2026-01-01*	*2026-01-01*	*2026-01-08*

In Python:

```
import dlt
from pyspark.sql.functions import col, expr

# ---------- Bronze: ingest raw CDC files incrementally ----------
@dlt.table(name="customers_bronze", comment="Raw CDC feed")
def customers_bronze():
    return (
        spark.readStream.format("cloudFiles")           # Auto Loader
        .option("cloudFiles.format", "json")
        .load("/Volumes/main/demo/landing/customers/")
    )

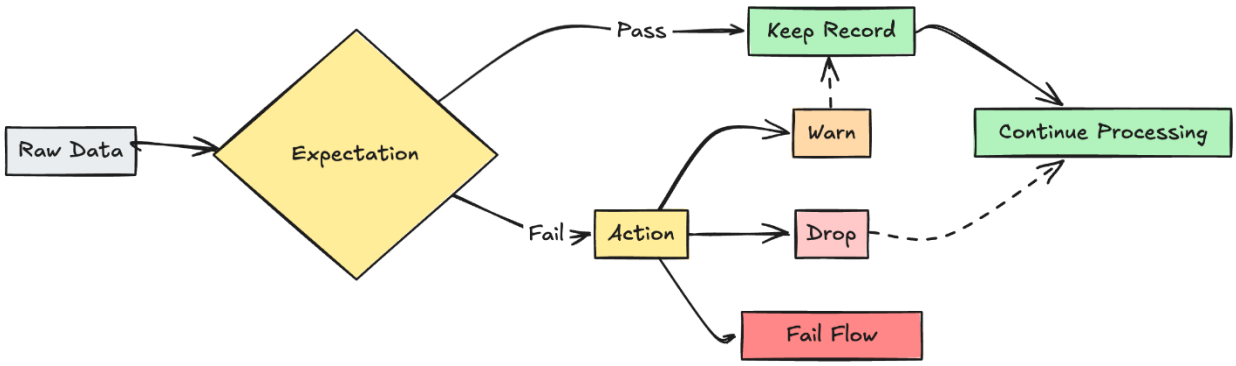
# ---------- Silver: light cleaning + data quality ----------
@dlt.view(name="customers_clean")
@dlt.expect_or_drop("valid_key", "customer_id IS NOT NULL")
@dlt.expect_or_drop("valid_ts", "updated_at IS NOT NULL")
def customers_clean():
    return dlt.read_stream("customers_bronze").select(
        "customer_id", "name", "city", "operation", "updated_at"
    )

# ---------- Gold: SCD Type 2 dimension ----------
dlt.create_streaming_table("dim_customer")              # target must be declared
first

dlt.apply_changes(
    target             = "dim_customer",
    source             = "customers_clean",
    keys               = ["customer_id"],               # business key
    sequence_by        = col("updated_at"),             # ordering column
    apply_as_deletes   = expr("operation = 'DELETE'"),  # optional
    except_column_list = ["operation"],                 # don't store these in
target
```

```
    stored_as_scd_type = 2                                  # <-- the key line
    # track_history_column_list = ["city"]                  # optional: only create new
                                                            # versions when these change
)
```

Manage data quality with pipeline expectations




```
@dp.table
@dp.expect("valid_customer_age", "age BETWEEN 0 AND 120")
def customers():
  return spark.readStream.table("datasets.samples.raw_customers")
```

```
CREATE OR REFRESH STREAMING TABLE customers(
  CONSTRAINT valid_customer_age EXPECT (age BETWEEN 0 AND 120)
) AS SELECT * FROM STREAM(datasets.samples.raw_customers);
```

All the constraints:

```sql
-- Simple constraint
CONSTRAINT non_negative_price EXPECT (price >= 0)

-- SQL functions
CONSTRAINT valid_date EXPECT (year(transaction_date) >= 2020)

-- CASE statements
CONSTRAINT valid_order_status EXPECT (
  CASE
    WHEN type = 'ORDER' THEN status IN ('PENDING', 'COMPLETED', 'CANCELLED')
    WHEN type = 'REFUND' THEN status IN ('PENDING', 'APPROVED', 'REJECTED')
    ELSE false
  END
)

-- Multiple constraints
CONSTRAINT non_negative_price EXPECT (price >= 0),
CONSTRAINT valid_purchase_date EXPECT (date <= current_date())

-- Complex business logic
CONSTRAINT valid_subscription_dates EXPECT (
  start_date <= end_date
  AND end_date <= current_date()
  AND start_date >= '2020-01-01'
)

-- Complex boolean logic
CONSTRAINT valid_order_state EXPECT (
  (status = 'ACTIVE' AND balance > 0)
  OR (status = 'PENDING' AND created_date > current_date() - INTERVAL 7 DAYS)
)
```

Action	SQL syntax	Python syntax	Result
warn (default)	`EXPECT`	`dp.expect`	Invalid records are written to the target.
drop	`EXPECT ... ON VIOLATION DROP ROW`	`dp.expect_or_drop`	Invalid records are dropped before data is written to the target. The count of drop alongside other dataset metrics.
fail	`EXPECT ... ON VIOLATION FAIL UPDATE`	`dp.expect_or_fail`	Invalid records prevent the update from succeeding. Manual intervention is requi

Retain invalid records : Retaining invalid records is the default behavior for expectations

```
@dp.expect("valid timestamp", "timestamp > '2012-01-01'")
```

```
CONSTRAINT valid_timestamp EXPECT (timestamp > '2012-01-01')
```

Drop invalid records : Use the expect_or_drop operator to prevent further processing of invalid records. Records that violate the expectation are dropped from the target dataset:

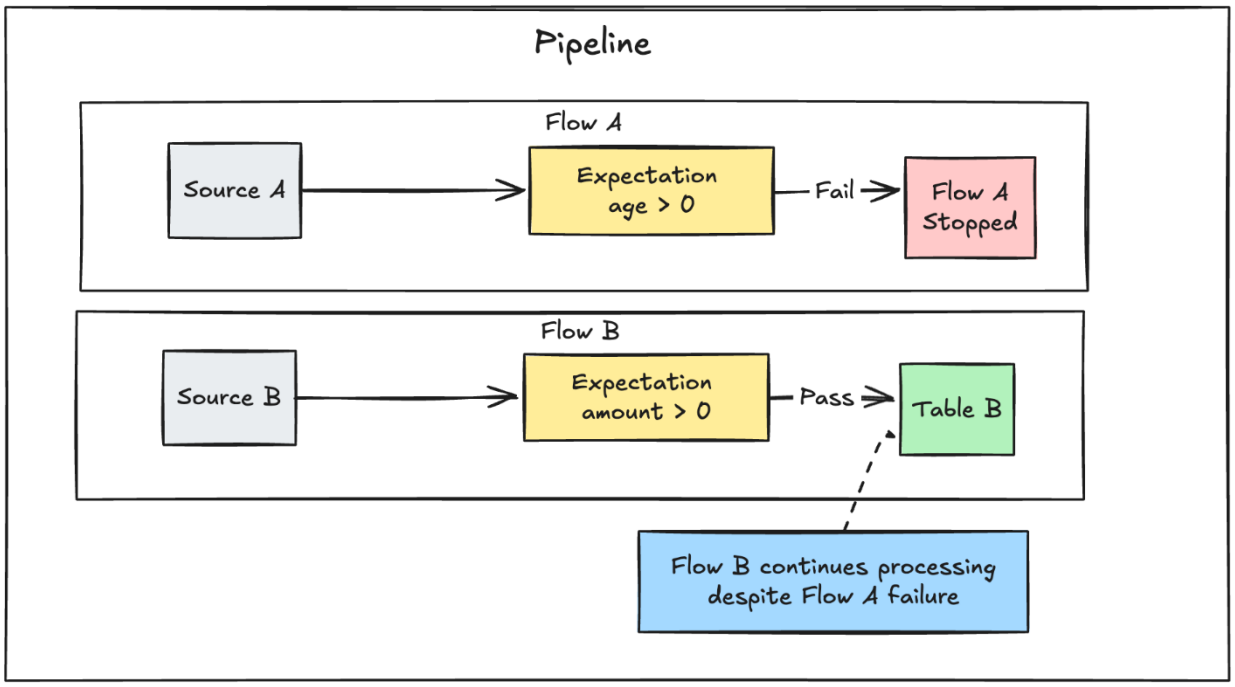
```
@dp.expect_or_drop("valid_current_page", "current_page_id IS NOT NULL AND
current_page_title IS NOT NULL")
```

```
CONSTRAINT valid_current_page EXPECT (current_page_id IS NOT NULL and
current_page_title IS NOT NULL) ON VIOLATION DROP ROW
```

Fail on invalid records : When invalid records are unacceptable, use the expect_or_fail operator to stop execution immediately when a record fails validation.

```
@dp.expect_or_fail("valid_count", "count > 0")
```

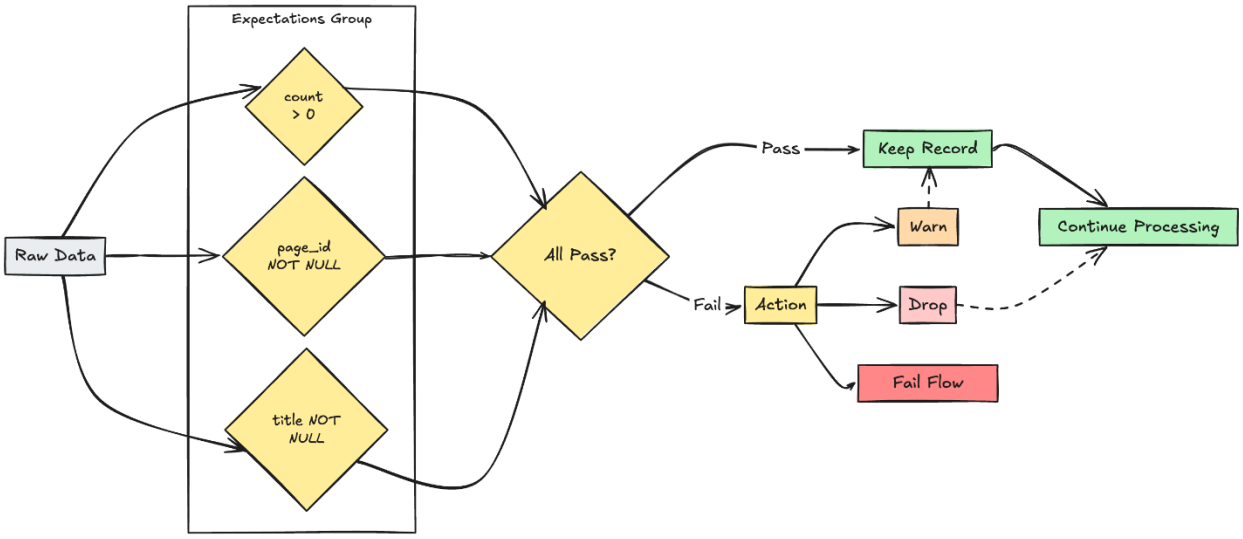
```
CONSTRAINT valid_count EXPECT (count > 0) ON VIOLATION FAIL UPDATE
```



In a triggered pipeline, the failure of a single flow does not cause other parallel flows to fail. In a continuous pipeline, an expectation failure stops the flow, all dependent flows, and the pipeline emits a message explaining why it stopped.

For more control over workflow orchestration when a validation fails, split the validation and downstream work into separate pipelines and coordinate them with control flow between the pipeline tasks.

Manage multiple expectations




```
valid_pages = {"valid_count": "count > 0", "valid_current_page": "current_page_id
IS NOT NULL AND current_page_title IS NOT NULL"}

@dp.table
@dp.expect_all(valid_pages)
def raw_data():
  # Create a raw dataset

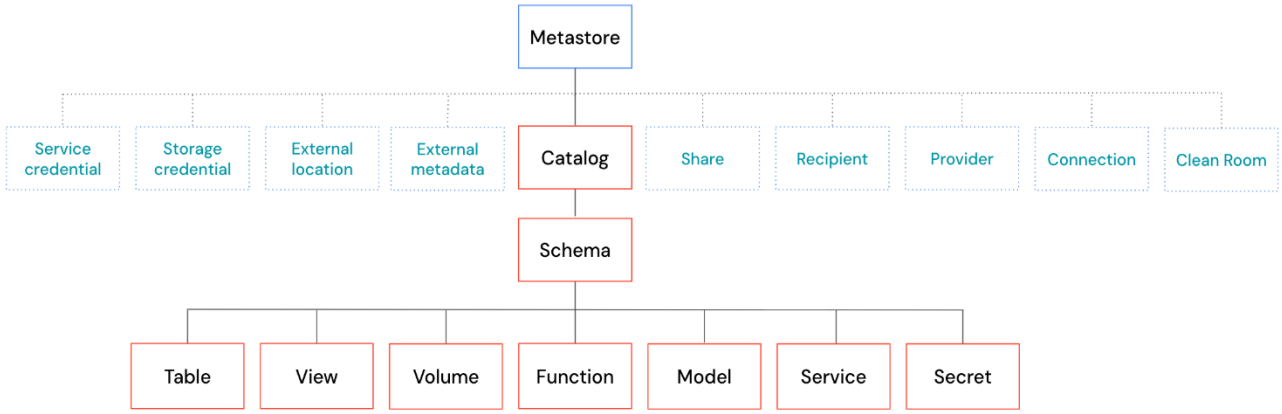
@dp.table
@dp.expect_all_or_drop(valid_pages)
def prepared_data():
  # Create a cleaned and prepared dataset

@dp.table
@dp.expect_all_or_fail(valid_pages)
def customer_facing_data():
  # Create cleaned and prepared to share the dataset
```

Expectations are not supported with AUTO CDC FROM SNAPSHOT.

Unity catalog:

unity catalog is a centralizws data catalog that provides access control, auditing,lineage,quality monitoring, and data discovery capabilities acorss Databricks workspaces.



The metastore is the top-level securable object. Within this metastore, your data assets live in a three-level namespace that defines its catalog, schema, and type of asset, such as table (catalog.schema.table).

- **Catalogs** are the top-level layer for your data assets. Catalogs exist directly under the metastore. They are used to organize your data and AI assets, typically by organizational units or software development lifecycle scopes.
    - **Schemas** exist within catalogs. They organize data and AI assets into categories that are more granular than catalogs. A schema may represent a single use case, project, or team sandbox.
        - **Tables** are collections of structured data organized by rows and columns.
        - **Views** are saved queries against other tables or views.
        - **Volumes** represent collections of unstructured data in cloud object storage.

- **Functions** are units of reusable logic that return a scalar value or set of rows.
- **Models** are versioned or unversioned AI models registered in Unity Catalog.
- **Services** are governed, invocable AI assets, such as model services and MCP services.
- **Secrets** store sensitive values, such as credentials and tokens, that you can govern in Unity Catalog and reference without exposing the value.

Storage credential

Within a metastore, a **storage credential** is a securable object that stores the authentication information required to access a specific path in cloud storage. The stored authentication method depends on the cloud provider: an IAM role on AWS, a service principal on Azure, or a service account on GCP.

Storage credentials are most commonly used as a building block for external locations, which pair a storage credential with a specific cloud storage path. A storage credential can also be used directly to create external tables.

To create a storage credential, a user needs the CREATE STORAGE CREDENTIAL privilege on the Unity Catalog metastore.

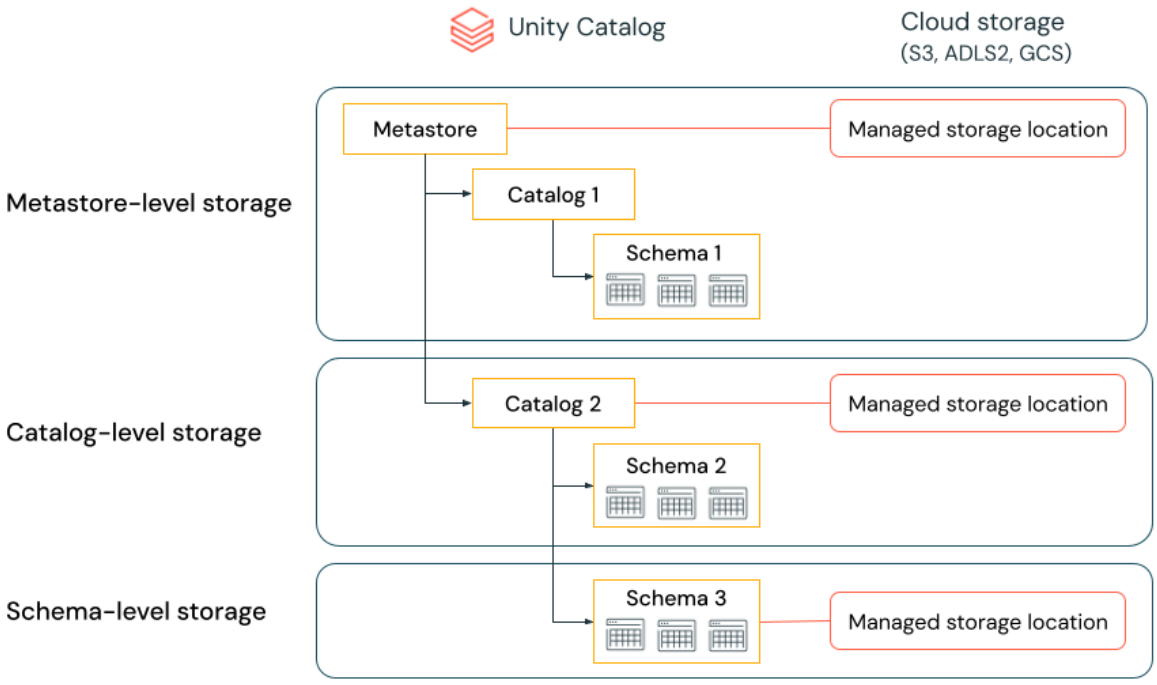
External location

Within a metastore, an **external location** is a securable object that pairs a storage credential with a cloud storage path. It governs access to a specific path in cloud storage.

To create an external location, a user needs the CREATE EXTERNAL LOCATION privilege on the Unity Catalog metastore.

After creating an external location, users need the READ FILES privilege to read files directly from the storage path, and the WRITE FILES privilege to write files. However, Databricks recommends managing cloud storage access through volumes and the READ VOLUME and WRITE VOLUME privileges rather than granting READ FILES and WRITE FILES directly on external locations.

Metastore Level storage:



Metastore is attached to the data lake.

Catalog is managed.

Schema is managed

And object is managed.

Here we are defininig the storage at metastore level only....

Now data for our objects [managed tables] will be saved in the metastore data lake..

```
%sql
CREATE CATALOG IF NOT EXISTS managed_catalog;
```

```
CREATE SCHEMA IF NOT EXISTS managed_catalog.managed_schema;
```

```
CREATE TABLE managed_catalog.managed_schema.managed_table
(
    id int,
    name string
)
insert into managed_catalog.managed_schema.managed_table values (1,'Naresh');
```

we can see our data in metastore container in storage account.

CATALOG LEVEL STORAGE:

Metastore is attached to the data lake.

Catalog is also attached to the data lake.

Schema is managed

And object is managed.

Here we are defininig the storage at catalog level....

```
%sql
CREATE CATALOG IF NOT EXISTS ext_catalog
MANAGED LOCATION 'abfss://metastore@databricksunitydemo.dfs.core.windows.net/catalog'
```

```
CREATE SCHEMA IF NOT EXISTS ext_catalog.ext_schema;

CREATE TABLE ext_catalog.ext_schema.managed_table
(
    id int,
    name string
)
insert into managed_catalog.managed_schema.managed_table values (1,'Naresh');
```

SCHEMA LEVEL STORAGE:

Metastore is attached to the data lake.

Catalog is also attached to the data lake.

Schema is also attached to the data lake.

And object is managed.

Here we are definining the storage at SCHEMA level....

```
%sql
CREATE SCHEMA IF NOT EXISTS ext_catalog.ext_schema
MANAGED LOCATION 'abfss://metastore@databricksunitydemo.dfs.core.windows.net/schema';

CREATE TABLE ext_catalog.ext_schema.managed_table
(
    id int,
    name string
)
insert into managed_catalog.managed_schema.managed_table values (1,'Naresh');
```

if we can define metastore, catalog, schema storage level it is always picks the nearest object. Ex the above scenario it will pick schema level storage level. Because of schema is nearest object.....

Managed vs External Tables:

Managed vs. External Tables

Characteristic	Managed Table	External Table
Storage cleanup on DROP	Automatic	Manual
Predictive Optimization	Supported	Not supported
Small file compaction	Background	Runs during ETL
Path management	Automatic	Manual
Best for	Internal pipelines	Data shared with external systems

```
%sql
DROP TABLE IF EXISTS catalog_name.schema_name.table_name;
```

```
-- Set a 30-day recovery period on a catalog
ALTER CATALOG my_catalog RETAIN DROPPED TO 30 DAYS;

-- Set a 7-day recovery period on a schema (overrides the catalog setting)
ALTER SCHEMA my_catalog.my_schema RETAIN DROPPED TO 7 DAYS;
```

You can also set the recovery period when creating a catalog or schema with the RETAIN DROPPED FOR clause:

```
%sql

CREATE CATALOG my_catalog RETAIN DROPPED FOR 30 DAYS;
CREATE SCHEMA my_catalog.my_schema RETAIN DROPPED FOR 7 DAYS;
```

```
SHOW TABLES DROPPED IN catalog_name.schema_name;
UNDROP TABLE catalog_name.schema_name.my_table;
UNDROP TABLE WITH ID '6ca7be55-8f58-47a7-85ee-7a59082fd17a';
```

EXTERNAL TABLES:

```
CREATE TABLE ext_catalog.ext_schema.ext_table
(
    id int,
    name string
)
location 'abfss://raw@databricksunitydemo.dfs.core.windows.net/ext_table'
insert into ext_catalog.ext_schema.ext_table values (1,'Naresh');
```

Here we are providing the object level or table level storage....

Volumes:

We have a csv file in storage account. Names.csv file now I want to read that file in databricks and govern that file...

```
%sql
CREATE EXTERNAL VOLUME IF NOT EXISTS ext_catalog.ext_schema.myExternalVolume
        COMMENT 'This is my example external volume'
        LOCATION 'abfss://raw@databricksunitydemo.dfs.core.windows.net/data'
```

Under data folder I have names.csv file...

If I want to read that file through unity catalog.

```
SELECT * FROM csv.`/volumnes/ext_catalog/ext_schema/myExternalVolume/names.csv`
```

For Masking:

```
REATE TABLE ext_catalog.ext_schema.employee
(
    id int,
    name string,
    salary int
)
insert into managed_catalog.managed_schema.managed_table values (1,'Naresh',3000);
```

```
%sql
CREATE FUNCTION ssn_mask(saalry int)
  RETURN CASE WHEN is_account_group_member('HumanResourceDept') THEN saalry  ELSE
'***-**-****' END;
```

```
%sql

ALTER TABLE employee ALTER COLUMN ssn SET MASK ssn_mask;  < we need to provide
function name>
```

WEEK21: Databricks Lakeflow Jobs

Decorators:

Decorators let you add extra behavior to a function, without changing the function's code.

A decorator is a function that takes another function as input and returns a new function.

Ex:

```
def my_dec(func):
    def wrapper():
        print("before function executes")
        func()
        print("after function executes")
    return wrapper
```

```
@my_dec
def greet():
    print("hello")
```

greet()

before function executes hello after function executes

I will return a dataframe after reading a csv file from cloud storage.

I want to use a decorator to convert this dataframe into a delta table.

```
def write_to_delta(func):
    def wrapper():
        df=func()
        table_name=func.__name__
        df.write.format("delta").saveAsTable("workspace.default."+table_name)
        print("delta table created")
    return wrapper
```

```python
@write_to_delta
def orders_df():
    df=(
        spark.read
        .format("csv")
        .option("header","true")
        .option("inferSchema","true")
        .load("/Volumes/dlt_flow/default/raw_data/orders/")
    )

    return df
```

```python
df=orders_df()
```

Declarative pipeline using python:

Utilities → utilis.py

```python
from pyspark.sql.functions import udf
from pyspark.sql.types import FloatType

def inr_to_usd(inr):
    """ Convert inr to USD (1 usd = 88.72)"""
    return inr/88.72
```

transformations.py

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import current_timestamp,col,sum as _sum
from utilities import utils

source_path= spark.conf.get("source")

#bronze : ingest orders
@dp.table(name="orders_bronze")
def orders_bronze():
    df =(
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format","csv")
        .option("cloudFiles.inferColumnTypes","true")
```

```
        .option("cloudFiles.schemaLocation", f"{source_path}/_schemas/orders")
        .load(f"{source_path}/orders")
    )
    return  (
        df.withColumn("filename",col("_metadata.file_path"))
          .withColumn("load_time",current_timestamp())
    )

#bronze : ingest customers
@dp.table(name="customers_bronze")
def customers_bronze():
    df =(
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format","csv")
        .option("cloudFiles.inferColumnTypes","true")
        .option("cloudFiles.schemaLocation", f"{source_path}/_schemas/customers")
        .load(f"{source_path}/customers")
    )
    return  (
        df.withColumn("filename",col("_metadata.file_path"))
          .withColumn("load_time",current_timestamp())
    )


# silver table : cleaned:
@dp.table(name="orders_silver_cleaned")
@dp.expect_or_drop("valid_order","order_id is not null")
@dp.expect_or_drop("valid_customer_id","customer_id is not null")
def orders_silver_cleaned():
    df=dp.readStream("orders_bronze")
    return (
        df.selectExpr(
                "orderid as order_id",
                "orderdate as order_date",
                "customerid as customer_id",
                "filename as file_name",
                "totalamount as total_amount",
                "status",
                "load_time"
        ).withColumn("total_amount_in_USD",utils.inr_to_usd(col("total_amount")))
    )

@dp.table(name="customers_silver_cleaned")
```

```python
@dp.expect_or_drop("valid_customer_id","customer_id is not null")
def customers_silver_cleaned():
    df=spark.readStream.table("customers_bronze")
    return (
        df.selectExpr(
                "customerid as customer_id",
                "customername as customer_name",
                "address as city",
                "dateofbirth as dob",
                "registrationdate as customer_since",
                "filename as file_name",
                "load_time"
        )
    )


# silver : SCD TYPE 2 FOR CUSTOMERS USING AUTO CDC
dp.create_streaming_table("customers_silver")
dp.create_auto_cdc_flow(
    target="customers_silver",
    source="customers_silver_cleaned",
    keys=["customer_id"],
    sequence_by=col("load_time"),
    stored_as_scd_type=2
)


#silver : SCD TYPE 1 FOR ORDERS USING AUTO CDC

dp.create_streaming_table("orders_silver")
dp.create_auto_cdc_flow(
    target="orders_silver",
    source="orders_silver_cleaned",
    keys=["order_id"],
    sequence_by=col("load_time"),
    stored_as_scd_type=1
)


# defining the gold layer: city wise  materialized views
@dp.materialized_view(name="city_wise_sales_gold")
def city_wise_sales_gold():
    orders=spark.read.table("orders_silver")
```

```
    customers=spark.read.table("customers_silver")
    return (
        orders.join(customers,"customer_id")
        .groupBy("city")
        .agg(_sum("total_amount").alias("total_sales"))
    )
```

Managed Connector

Is a pre-built, fully-managed tool that automatically ingests data from external sources—such as enterprise SaaS apps or relational databases—into Delta streaming tables with minimal manual setup..

Azure sql database

- Orders
- Customers

CT (change tracking)

CDC (change data capture)

Feature	Change Tracking (CT) 🔄	Change Data Capture (CDC) 🪙
What it records	Only **that** a row changed, and the primary key of that row.	The **actual data** that changed (the before and after values).
Storage Mechanism	Lightweight internal tracking tables.	Reads the database **Transaction Log** (asynchronous).
Performance Impact	Minimal impact on the source database.	Slightly higher than CT, but still highly efficient.
History Tracking	No history. It only retains the current net change.	Full historical audit trail of every single change over time.

Best Used For	Two-way data synchronization and basic cache refreshing.	Data Warehousing, Event Streaming, and complex ETL pipelines.

Connection to the database – credentials (unity catalog)

Get the data from the database to your volume (staging area) -- gateway

From volume (staging area) data is ingested in your delta tables – pipeline.

This gateway (classic compute) – will keep running.....

Ingestion pipeline (batch) – serverless (volume and bring it to delta lake)

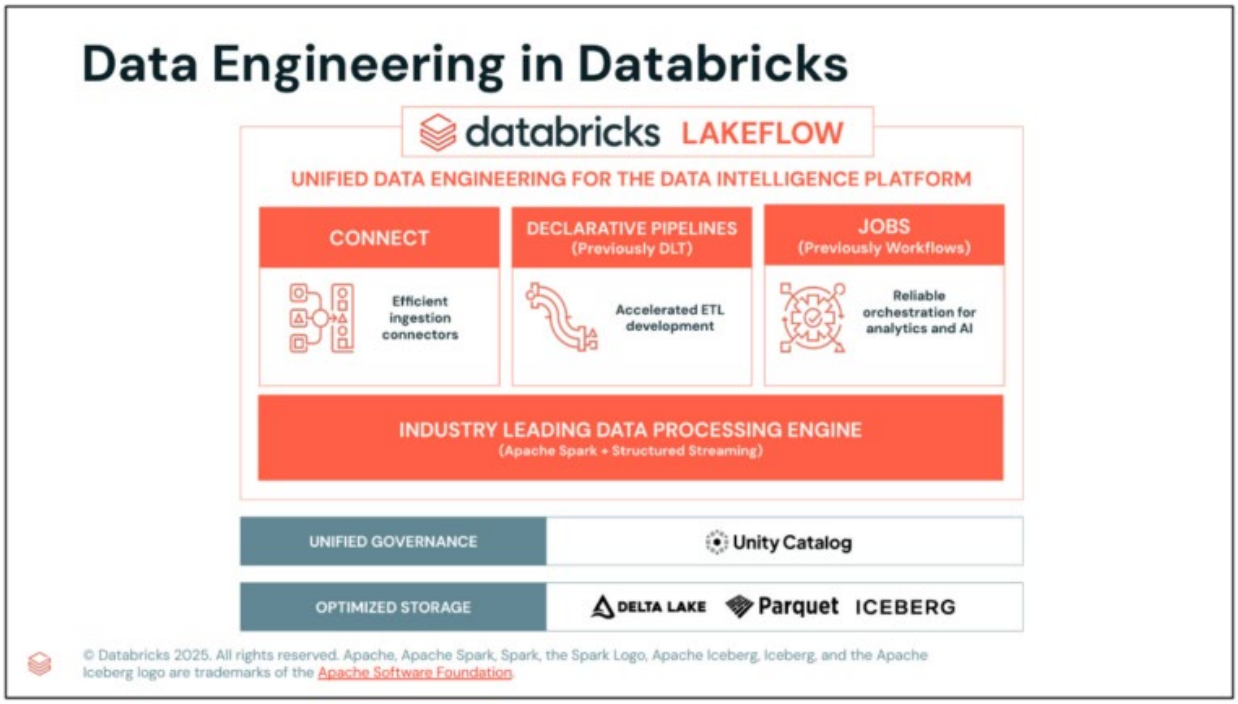
Jobs and pipeline → Ingestion pipeline → SQL server → create connection....

Lakeflow Jobs:

Lakeflow connect (for ingestion)

Lakeflow declarative pipelines (ETL)

Lakeflow Jobs (for Orchestration)



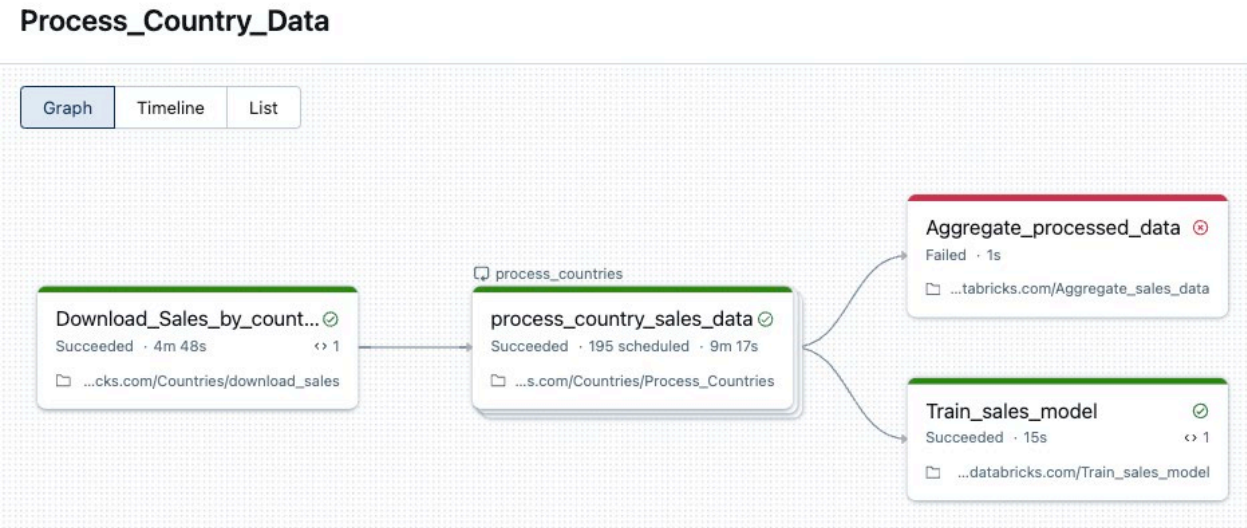
Airflow , ADF we have some other orchestration tools are available....

The challenge with 3rd party orchestration tools,

- Not unified with Delta lake, integration becomes hard.
- High infrastructure overhead.
- You must securely pass, store and rotate cloud credentials, API tokes and SSh to allow external orchestration to communicate with databricks..

What is a Job here ?

Task – is a single unite of work within a job. It can execute specific workloads such as executig a notebook,scripts,sql query etc...



The entire flow we can called as job and each box is (it is doing some kind of trasformation or SQL query or script) anything is called as task.. each box is called as task...

Task can be :

- ingesting customer from raw table.
- ingesting customer from a folder.

Think job as a container which holds the entire workflow.

When you create a job you can set specific configurations for each prticular task.

- Defining the path to the code.
- Setting parameters.
- Enable notifications
- Configure the retry policies.

TakeWays:

- Lakeflow Jobs - consist of one or more tasks (notebooks,dlt,sql files, python scripts)
- control flows can be established between tasks (sequential, parallel, conditonal, for each)
- Task 1 (execution was done) → Task 2 (after Task1 execution Task2 begins).
- Jobs supports different types of triggers (scheduled, file arrival trigger, table trigger, continuous trigger, manual trigger).

Medallion Architecture Basics

Bronze → Silver → Gold

Scenario:

1. In SQL server we have customers table → Then We have Lakeflow connect → then bringing to the Delta table (Customers Raw table).
2. Orders (table in sql server) → a third party tool is bringing incremental orders and putting in a csv file within a folder in ADLS gen2 (every Day we are getting a new file)...

   Ex: abfss://dataset@lakeflowstorageaccount.dfs.core.windows.net/orders

So 2 sources of data we have

1. Customers data (lakeflow connect is getting the data)
2. Orders (the csv file in orders folder)

After connecting lakeflow connect we have data like below:

```
CREATE SCHEMA IF NOT EXISTS ingest_schema;

-- CREATE TABLE
CREATE TABLE IF NOT EXISTS ingest_schema.customers(
    CustomerID INT,
    CustomerName STRING,
    ContactNumber STRING,
    Email STRING,
    Address STRING,
```

```
    DateOfBirth DATE,
    RegistrationDate DATE,
    EffectiveStartDate TIMESTAMP,
    EffectiveEndDate TIMESTAMP
) USING DELTA
TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true');

-- CURRENT VERSIONS (your original data; EffectiveEndDate = NULL)
INSERT INTO ingest_schema.customers VALUES
('1','Tara Yadav','9682667034','diya.desai@hotmail.com','Bhopal','1980-01-01','2020-
01-01', current_timestamp(), NULL),
('2','Vivaan Joshi','9367873685','nikhil.patel@gmail.com','Visakhapatnam','1980-01-
08','2020-01-04', current_timestamp(), NULL),
```

customers_bronze_ingestion.sql

```
--  STEP1 : ENSURE bronze schema exists
CREATE SCHEMA IF NOT EXISTS bronze_schema;

-- step2:
CREATE OR REPLACE TABLE bronze_schema.customers_bronze
As select * from ingest_schema.customers;
```

customer_silver_cleaned (Notebook)

```
from pyspark.sql.functions import col
spark.sql("CREATE SCHEMA IF NOT EXISTS silver_schema")

customers_cleaned=(spark.table("bronze_schema.customers_bronze")
.filter(col("EffectiveEndDate").isNull()) # keep on current records not historical
records.
.select(
    col("CustomerId").alias("customer_id"),
    col("CustomerName").alias("customer_name"),
    col("ContactNumber").alias("contact_numer"),
    col("Email").alias("email"),
    col("Address").alias("city"),
    col("DateOfBirth").alias("dob"),
    col("RegistrationDate").alias("registration_date")
))
```

```
customers_cleaned.write.mode("overwrite").saveAsTable("silver_schema.customers_cleaned")
print("customer_cleaned_complete:silver_schema.customers_cleaned ")
```

customer_silver_enriched (Notebook)

customers_silver_enriched : we are adding json file to match city with respective state

```
from pyspark.sql.functions import col
spark.sql("CREATE SCHEMA IF NOT EXISTS silver_schema")

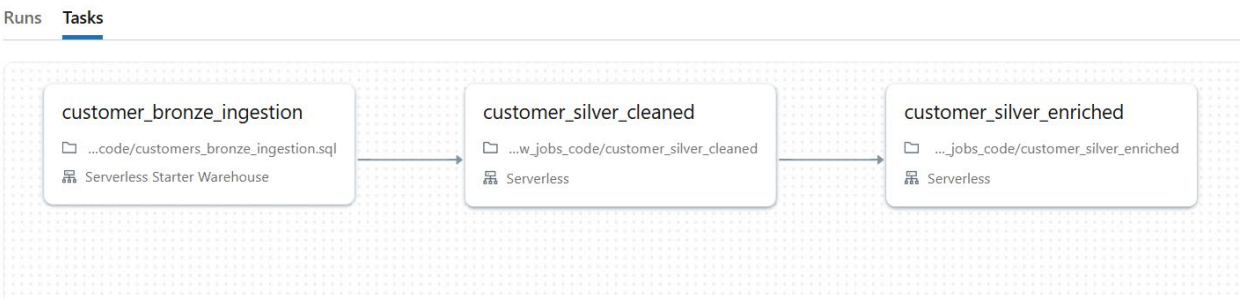
customers_df=spark.table("silver_schema.customers_cleaned")
lookup_path="/Volumes/workspace/default/lookup_path/lookup_path/"
city_state_df=spark.read.format("json").load(lookup_path)

# Now I want to perform a join

customers_enriched = customers_df.join(city_state_df,customers_df.city == city_state_df.city,"left").drop(city_state_df.city) # once join chesaka city avasaram ledhu

# write into a table

customers_enriched.write.format("delta").mode("overwrite").saveAsTable("silver_schema.customers_enriched")
```



Lets work on orders:

orders_bronze_ingestion (type: Notebook)

```
%md
###  here we are ingesting orders files from external source
```

```
orders_csv_file_path="/Volumes/workspace/default/raw_files/orders/"
orders_df=(spark.read.format("csv")
             .option("header","true")
             .option("inferSchema","true")
              .load(orders_csv_file_path))
```

```
orders_df.write.format("delta").mode("overwrite").saveAsTable("bronze_schema.orders_b
ronze")
```

orders_silver_cleaned (type : Notebook)

```
from pyspark.sql.functions import col,to_date,row_number
from pyspark.sql.window import Window

#step1: Ensure silver schema exists
spark.sql("CREATE SCHEMA IF NOT EXISTS silver_schema")

#load bronze orders
bronze_orders=spark.table("bronze_schema.orders_bronze")

#step3: Rename and cast columns
orders_cleaned = (
    bronze_orders.select (
        col("OrderId").alias("order_id"),
        to_date(col("OrderDate")).alias("order_date"),
        col("CustomerID").alias("customer_id"),
        col("TotalAmount").cast("double").alias("total_amount"),
        col("Status").alias("status")
    )
)

#step4: write to silver layer
orders_cleaned.write.format("delta").mode("overwrite").saveAsTable("silver_schema.ord
ers_cleaned")
print("Silver cleaned complete: silver_schema.orders_cleaned")
```

Identify the duplicates using window functions

```
w=Window.partitionBy("order_id").orderBy(col("order_date").desc())
orders_with_rn=orders_cleaned.withColumn("rn",row_number().over(w))
```

```
total_cleaned=orders_cleaned.count()
total_dedupe = orders_with_rn.filter(col("rn")==1).count()
duplicate_exists = total_cleaned != total_dedupe
```

Task values

```
dbutils.jobs.taskValues.set(key="has_duplicates",value=duplicate_exists)
```

Original expression

```
{{tasks.orders_silver_cleaned.values.has_duplicates}} == "true"
```

Resolved expression

```
"true" == "true"
```

Expression outcome

```
true
```

orders_dedupe (type: Notebook)

```
from pyspark.sql.functions import col,to_date,row_number
from pyspark.sql.window import Window

#step1: Ensure silver schema exists
spark.sql("CREATE SCHEMA IF NOT EXISTS silver_schema")

orders_cleaned=spark.table("silver_schema.orders_cleaned")
```

```
w=Window.partitionBy("order_id").orderBy(col("order_date").desc())
orders_with_rn=orders_cleaned.withColumn("rn",row_number().over(w))
```

here we are doing order by order_date desc so the latest records comes on top then droping rn column -- zero duplicates

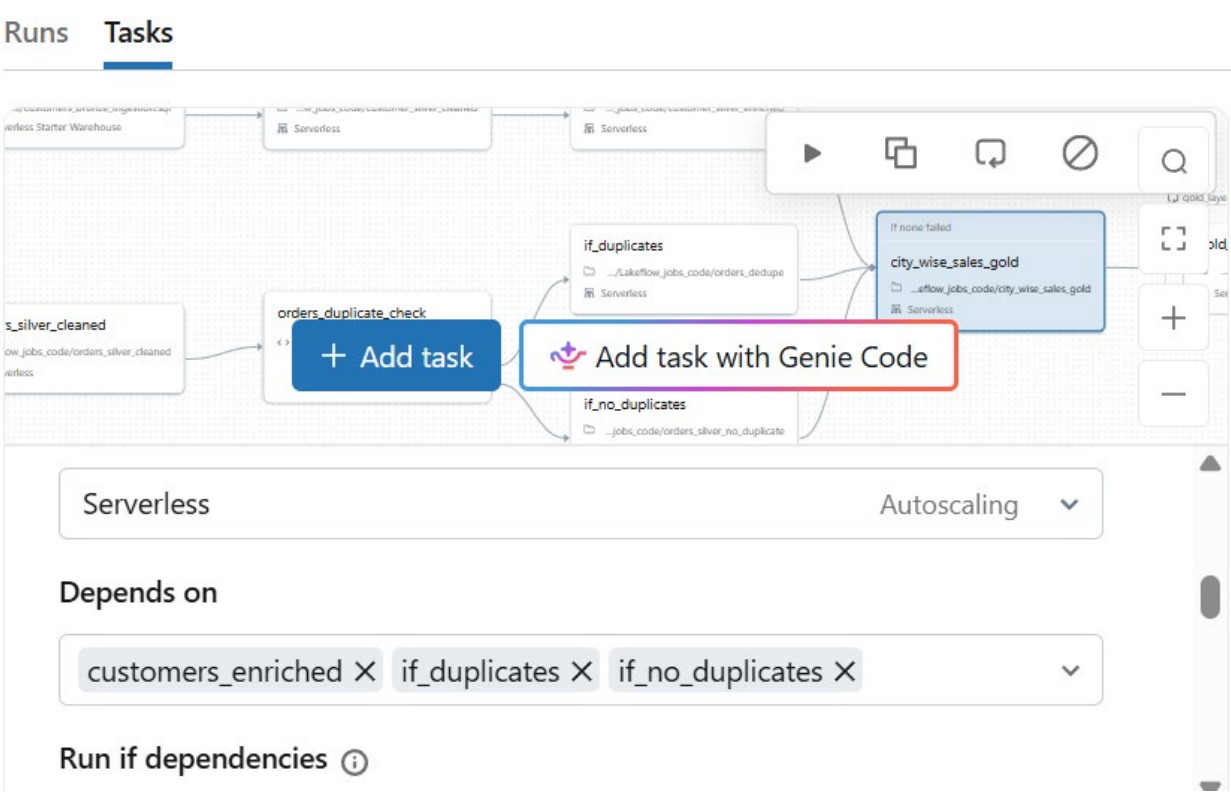
```
orders_dedupe=orders_with_rn.filter(col("rn")==1).drop("rn")
orders_dedupe.write.format("delta").mode("overwrite").saveAsTable("silver_schema.orders_dedupe")
```

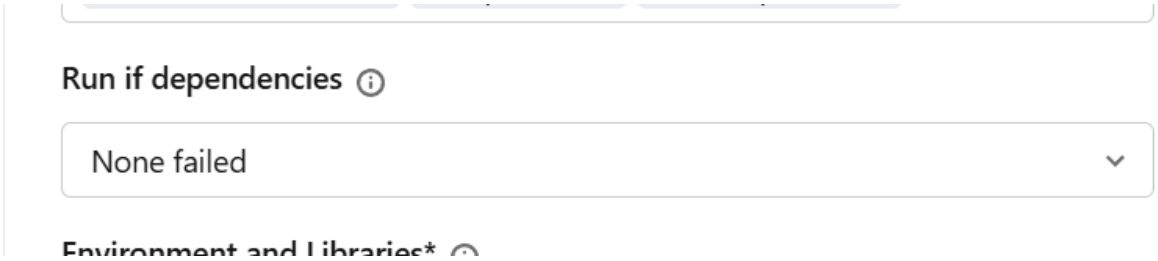
orders_silver_no_duplicate (type: Notebook)

```
#step1: Ensure silver schema exists
spark.sql("CREATE SCHEMA IF NOT EXISTS silver_schema")

orders_cleaned=spark.table("silver_schema.orders_cleaned")
orders_cleaned.write.format("delta").mode("overwrite").saveAsTable("silver_schema.orders_dedupe")
```

city_wise_sales_gold (type: Notebook)






```
from pyspark.sql.functions import sum,current_timestamp,col

#step1: Ensure gold schema exists
spark.sql("CREATE SCHEMA IF NOT EXISTS gold_schema")

#step2: load the silver tables
customers=spark.table("silver_schema.customers_enriched")
orders=spark.table("silver_schema.orders_dedupe")

# I want to join the both the tables
```

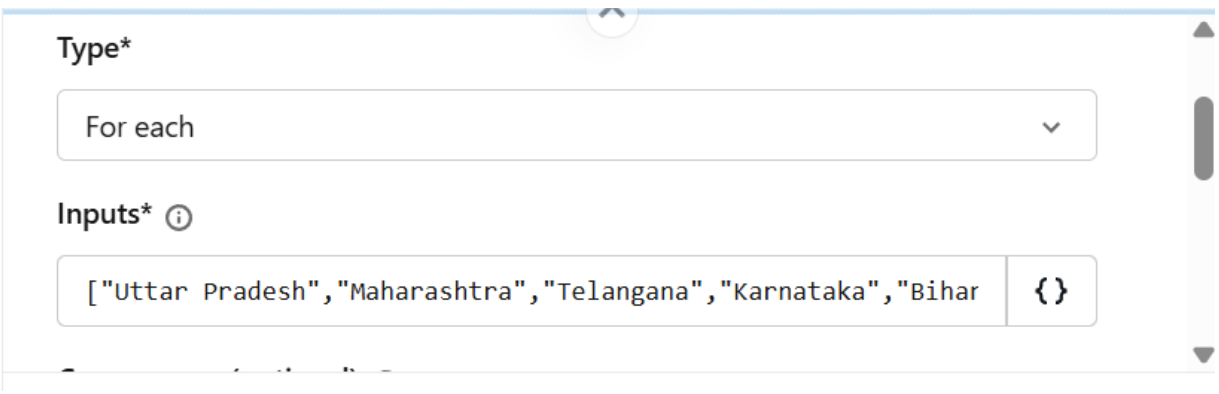
```
city_wise_sales=(orders.join(customers,orders.customer_id==customers.customer_id,"inner")
.filter((col("city").isNotNull()) & (col("state").isNotNull()))
.groupBy("state","city")
.agg(sum("total_amount").alias("total_sales"))
)

# I want to write to the gold table
city_wise_sales.write.mode("overwrite").saveAsTable("gold_schema.city_wise_sales")
print("gold layer completed")


# city   state   total_sales
# A       B       1233
```

For example we have 5 states for each state I want to create 5 different tables in that case I want to use for each demo:

Here we need to pass inputs like ["Uttar Pradesh","Maharashtra", "Telangana"]




```
from pyspark.sql.functions import sum,current_timestamp,col
```

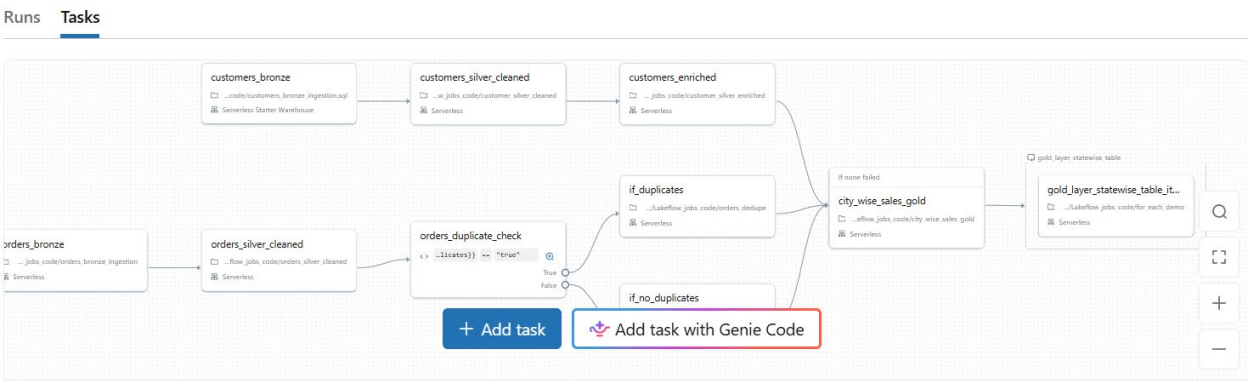
```
# I want to read the input
state_param=dbutils.widgets.get("state")
print(f"Procesing state: {state_param}")

# Safe-proof the string: lower case and replace spaces with underscores
safe_state_name = state_param.lower().replace(" ", "_")

# loading the gold table
gold_df=spark.table("gold_schema.city_wise_sales")
state_df=gold_df.filter(col("state") == state_param)
state_final_df=state_df.withColumn("large_city",
                    (col("total_sales")>1000).cast("boolean")
                    )

state_final_df.write.format("delta").option("mergeSchema","true").mode("overwrite").s
aveAsTable(f"gold_schema.state_sales_{safe_state_name.lower()}")
```

the final graph is:



WEEK22:

Medallion Architecture & Workflow Orchestration:

Introduction to Incremental Data Processing & Medallion Architecture:

In the last week we covered how to build end to end lakeflow Job,

Medalion architecture

Notebook

SQL

If / else

For each

now what if you want to make the things incremental ? for Lakeflow jobs:

Ideally you will use lakeflow Declarative pipelines, where things will be taken care automatically.

Lets say in your orders folder 100 files are already there, and one new file comes so in this case we need to take care of the only new file only. Not to reprocess those 100 files.

In your customer raw table if 1 million records are there, and 20 new records are coming inserts, updates, deleted.

Ideally you should not process everything from beginning.

Orders→ folder

Copy into / auto loader

Bronze temp table you captured all the new records

You will put it to bronze table

TBLPROPERTIES('delta.enableChangeDataFeed'='true')

How do you understand what are the new changes are there ?

table_changes('orders_bronze',2) # after version 2

merge from source (orders_bronze) to target table (orders_silver_cleaned)

for customers ,make sure

TBLPROPERTIES('delta.enableChangeDataFeed'='true')

So that you can get only the changed data after a certain version.

table_changes('customers_raw',1) #after version 1

merge into the bronze.

4 videos

Medalion architecture → Delta architecture

For delete (no deletion vectors)

Making Lakeflow jobs incremental:

Delta Architecture (Medalion architecture) – Session 1:

We mentioned that your lakehouse architecture can handle different usecase..

Delta Architecture

Bronze → Silver → Gold

Delta Architecture helps us to improve the quality of data.

Bronze →

- Raw data
- Single source of truth
- Data can be stored in any format.

Silver →

- Cleaned and filtered data
- In delta format
- It can serve machine learning and data science use case.

Gold →

- Highly aggregated, business specific data
- Optimized query layer.
- BI/ reporting

We can even have a platium layer if the business demands that...

File1:

Order_id, order_date, customer_id, order_status

1,2013-07-25 00:00:00.0,11599,CLOSED

3,2013-07-25 00:00:00.0,11599,COMPLETE

4,2013-07-25 00:00:00.0,11599,PENDING

Add file1

Update orders set order_status='CLOSED' where order_id=3;

File2:

3,2013-07-25 00:00:00.0,11599,CLOSED

1,2013-07-25 00:00:00.0,11599,CLOSED

4,2013-07-25 00:00:00.0,11599,PENDING

And logs will be :

Remove file1

Add file2

What if we ***delete*** where order_id=4;

File3:

3,2013-07-25 00:00:00.0,11599,CLOSED

1,2013-07-25 00:00:00.0,11599,CLOSED

< order_id =4 is deleted >

And logs would be:

Remove file2

Add file3

INSERT:

What if we **insert** a record:

Insert into orders values (11,'2023-09-09',918,'PAYMENT_REVIEW')

File4:

11,’2023-09-09’,918,’PAYMENT_REVIEW

So the log would be:

Add file4

Change data feed (CDC):

If you want to do audit for example which rows are changed / added / delted in your table you can get to know...

To merge the incremental changes...

```
%sql
create catalog if not exists cdc_learning;
```

```
%sql
use catalog cdc_learning;
```

```
%sql
CREATE TABLE cdc_learning.cdc_learning.orders (
  order_id INT,
  order_date DATE,
  customer_id INT,
  total_amount DECIMAL(10, 2),
  order_status STRING
)
USING delta
TBLPROPERTIES(delta.enableChangeDataFeed=true)
```

2.we can alter the change data feed property:

alter table table_name set TBLPROPERTIES (delta.enableChangeDataFeed =true)

3. set spark.databricks.delta.propertys.defaults.enableChangeDataFeed=true (from that time onwards every table have this feature enabled by default)

```
%sql
describe history cdc_learning.cdc_learning.orders;
```

OPERATION:

CREATE TABLE

```
%sql
INSERT INTO cdc_learning.cdc_learning.orders (order_id, order_date, customer_id,
total_amount, order_status) VALUES
(1, '2023-01-01', 25, 6202.83, 'Cancelled'),
(2, '2023-01-02', 32, 9871.87, 'Cancelled'),
(3, '2023-01-03', 78, 6814.71, 'Completed'),
(4, '2023-01-04', 89, 4972.72, 'Pending'),
(5, '2023-01-05', 91, 5150.90, 'Pending');
```

```
%sql
describe history cdc_learning.cdc_learning.orders;
```

operation

WRITE

CREATE TABLE

```
%sql
select * from table_changes('cdc_learning.cdc_learning.orders',1);

-- after version 1 we inserted 5 records
```



	_change_type	_commit_version	_commit_timestamp
	insert	1	2026-10-11T06:02:35.000+00:00
	insert	1	2026-10-11T06:02:35.000+00:00
	insert	1	2026-10-11T06:02:35.000+00:00

delete

```
%sql
delete from cdc_learning.cdc_learning.orders where order_id=3;
```

```
%sql
select * from table_changes('cdc_learning.cdc_learning.orders',2);
 -- version 2 tharavath we did only one delete
```

order_status	_change_type	_commit_version	_commit_timestamp
Completed	delete	3	2026-10-11T06:21:01.000+00:00

UPDATE

```
%sql
update cdc_learning.cdc_learning.orders set order_status='COMPLETE' WHERE order_id=4;
```

```
%sql
describe history cdc_learning.cdc_learning.orders;
```

operation

OPTIMIZE

UPDATE

OPTIMIZE

DELETE

SET TBLPROPERTIES

WRITE

CREATE TABLE

changes are logged in _change_data (only update,delete and merge)

what is the usecase for this CDC or change data feed ?

1. auditing

2. it will help you incremental merges among the table (example: bronze to silver)

Internals of Delta Architecture:

Bronze → Silver → Gold

- It helps to improve the quality of data.
- Serves different usecase
- Medallian architecture.

```
%sql
create schema if not exists retaildb;
```

```sql
%sql
CREATE TABLE cdc_learning.retaildb.orders_bronze (
  order_id INT,
  order_date STRING,
  customer_id INT,
  total_amount DECIMAL(10, 2),
  order_status STRING,
  filename string,
  created_on timestamp
)
USING delta
TBLPROPERTIES(delta.enableChangeDataFeed=true);
```

```sql
%sql
copy into cdc_learning.retaildb.orders_bronze from
(
    select
        orderid::int as order_id,
        orderdate::string as order_date,
        customerid:: int as customer_id,
        totalamount:: decimal(10,2) as total_amount,
        status::string as order_status,
         _metadata.file_path as filename,
        current_timestamp() as created_on
        from '/Volumes/workspace/default/raw_files/orders/'
)
fileformat=CSV
format_options('header'='true')

--  If you try to run above command again it won't impact.. the copy command will track all the records what are insereted or what are not inserted
```

```sql
%sql
select * from cdc_learning.retaildb.orders_bronze;
```

```sql
%sql
describe history cdc_learning.retaildb.orders_bronze;
```

operation

COPY INTO

CREATE TABLE

```
%sql
select * from table_changes('cdc_learning.retaildb.orders_bronze',1)
```

SILVER

Spark SQL does not allow temporary views to be qualified with a specific database or catalog name (like cdc_learning.retaildb). Temporary views are scoped strictly to your current session, so they cannot belong to a permanent schema.

```
%sql
create or replace temporary view orders_bronze_changes
as
select * from  table_changes('cdc_learning.retaildb.orders_bronze',2) where
customer_id >0 is not null and order_id> 0 and order_status in
('Cancelled','Completed','Pending')
```

```
%sql
CREATE TABLE cdc_learning.retaildb.orders_silver (
  order_id INT,
  order_date DATE,
  customer_id INT,
  total_amount DECIMAL(10, 2),
  order_status STRING,
  order_year int GENERATED ALWAYS AS (YEAR(order_date)),
  order_month int GENERATED ALWAYS AS (MONTH(order_date)),
  created_on timestamp,
  modified_on timestamp
)
USING delta
TBLPROPERTIES(delta.enableChangeDataFeed=true);
```

Merge command from bronze to silver:

```
%sql
merge into cdc_learning.retaildb.orders_silver tgt
using orders_bronze_changes src
on tgt.order_id=src.order_id
when matched then update set
tgt.order_status=src.order_status,tgt.customer_id=src.customer_id,tgt.modified_on=cur
rent_timestamp()
when not matched then insert
(order_id,order_date,customer_id,total_amount,order_status,created_on,modified_on)
values
(order_id,order_date,customer_id,total_amount,order_status,current_timestamp(),curren
t_timestamp());
```

GOLD

```
%sql
CREATE TABLE cdc_learning.retaildb.orders_gold (
  customer_id INT,
  order_status STRING,
  order_year int,
  total_no_orders int
)
USING delta
TBLPROPERTIES(delta.enableChangeDataFeed=true);
```

```
Problem statement:

for each customer for the particular year how many orders placed

100, 2013, completed  --> 5
100,2013,CLOSED --> 2
100,2014, complted -->2
```

```sql
%sql
insert overwrite table cdc_learning.retaildb.orders_gold
select customer_id,order_status,order_year,count(order_id) as total_no_orders
from cdc_learning.retaildb.orders_silver
group by customer_id,order_status,order_year
```

lakeflow job continuation:

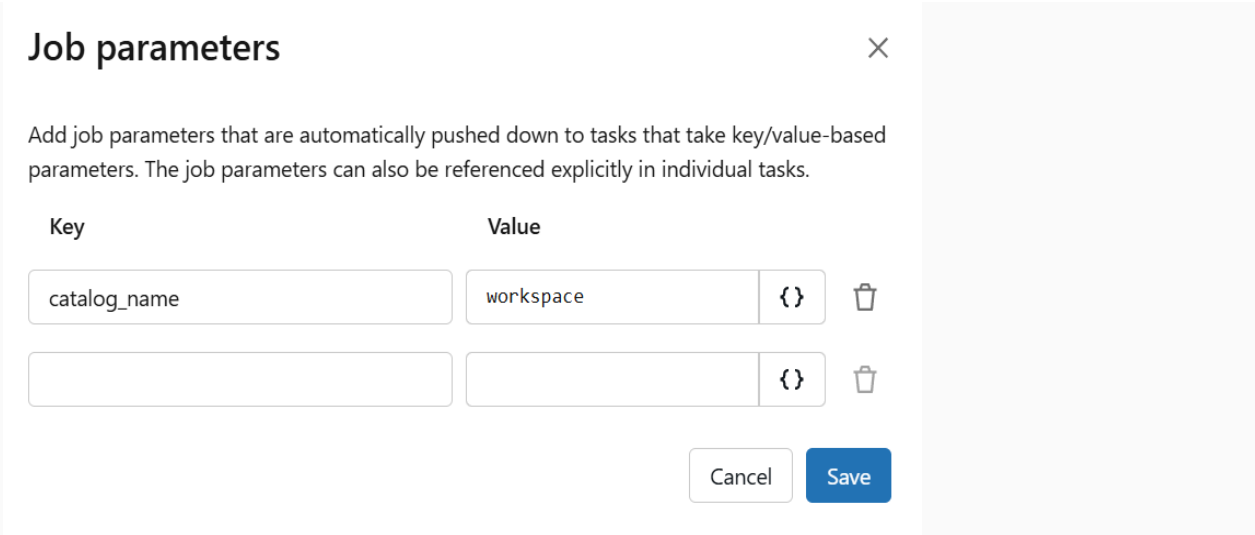
Job Parameters and Task Parameters:

Job Parameters (A key value pair defined at the job level and pushed down to all the tasks).

Task Parameters (a key value pair defined at the task level)

Whether you are retrieving a job parameter or task paramter.

Dbutils.widgets.get("catalog_name")



```
catalog_value=dbutils.widgets.get("catalog_name",”workspace”)
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog_value}.silver_schema")
```

```
# 1. Explicitly declare the widget (acts as a default value if missing)
dbutils.widgets.text("catalog_name", "development_catalog")

# 2. Safely retrieve the value
catalog_value = dbutils.widgets.get("catalog_name")

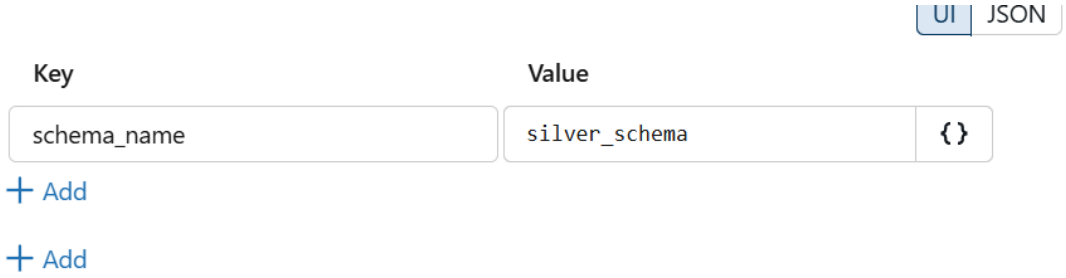
# 3. Execute your SQL query
spark.sql(f"DROP TABLE IF EXISTS {catalog_value}.silver_schema.customers_cleaned")
```

1. Job-Level Parameters (Shared Settings)

Pass values here if **multiple tasks** in your workflow need access to the exact same information. If you change a job-level parameter, every task using it gets updated automatically.

- **environment**: Specifies the deployment stage (e.g., dev, qa, stage, prod). This is often used to dynamically construct file paths or resource names.
- **catalog_name**: The target Unity Catalog name (e.g., mfg_dev_catalog, sales_prod_catalog).
- **business_unit / project_name**: Used for tagging data or organizing outputs (e.g., finance, marketing).
- **processing_date**: Used when backfilling or processing data for a specific historical date (e.g., 2026-10-13).
- **storage_account_path**: The root URI of your cloud landing zone or data lake storage (e.g., abfss://container@storage.dfs.core.windows.net/).

Task Parameters:



Task-Level Parameters (Specific Configurations)

Pass values here if the setting only matters to **one specific step** in the workflow sequence.

- **source_table / target_table**: The specific tables a task reads from or writes to (e.g., raw_transactions or customers_cleaned).
- **schema_name**: The layer of your architecture this specific task targets (e.g., bronze_schema, silver_schema, gold_schema).
- **file_format**: The format of the file being processed by an ingestion task (e.g., csv, parquet, json).
- **load_type**: Dictates how the data should be written (e.g., append, overwrite, upsert).
- **alert_email**: A specific email contact for a data-quality validation task if it fails.

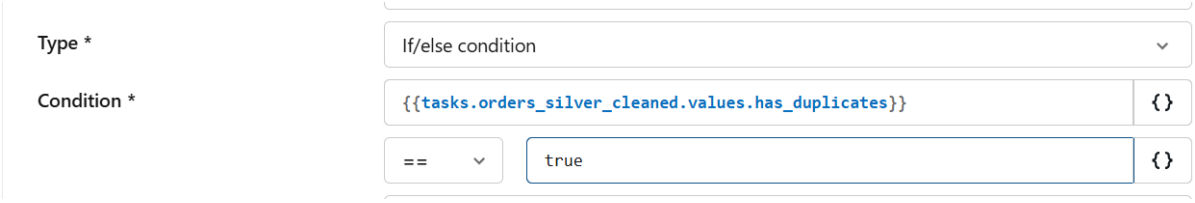
***Task Values & Dynamic Value References*:**

if a task want to set a parameter , so that other task can retieve that dynamically.

```
total_cleaned=orders_cleaned.count()
total_dedupe = orders_with_rn.filter(col("rn")==1).count()
duplicate_exists = total_cleaned != total_dedupe
```

dbutils.jobs.taskValues.set(key="has_duplicates",value=duplicate_exists)

dbutils.jobs.taskValues.get(taskKey=”orders_silver_cleaned”, key="has_duplicates")



task1 > task2

task1:

Dbutils.jobs.taskValues.set (

Key =”catalog_name”

Value=”databrickslearning”

)

task2:

Dbutils.jobs.taskValues.get (

taskkey=’task1’,

Key= “catalog_name”

)

Task values represent a more advance pattern than paramters they are computed at run time and enable dynamic communication between the tasks.

Dynamic Value Reference:

Allow referencing values at runtime from job and task context.

```python
import random

# Imagine this queries a directory or database to get a file count
computed_files = random.randint(10, 100)
print(f"Discovered {computed_files} files to process.")

# Push this value to the Databricks Workflow context
dbutils.jobs.taskValues.set(key="file_count", value=computed_files)
```

Parameter Key	Parameter Value (Dynamic Reference)	What it does
active_job_id	{{job.id}}	Injects the unique Databricks Job ID
execution_date	{{job.start_time.iso_date}}	Injects the current date (e.g., 2026-10-14)
files_to_process	{{tasks.Get_Metadata.values.file_count}}	Dynamically fetches the integer set by Task A

```python
 #1. Fetch the values passed through the dynamic references
job_id     = dbutils.widgets.get("active_job_id")
exec_date  = dbutils.widgets.get("execution_date")
file_count = dbutils.widgets.get("files_to_process")

# 2. Use them in your application logic
print(f"Running Job ID: {job_id}")
print(f"Processing data for date: {exec_date}")
print(f"Total files scheduled for ingestion: {file_count}")

# Example logic using the variables
target_directory = f"s3://my-bucket/archive/{exec_date}/"
print(f"Data will be written to {target_directory}")
```

{{ }}

{{job.start_time.day}}

{{task.name}}

{{task.<taskname>.values.<myvalue}}

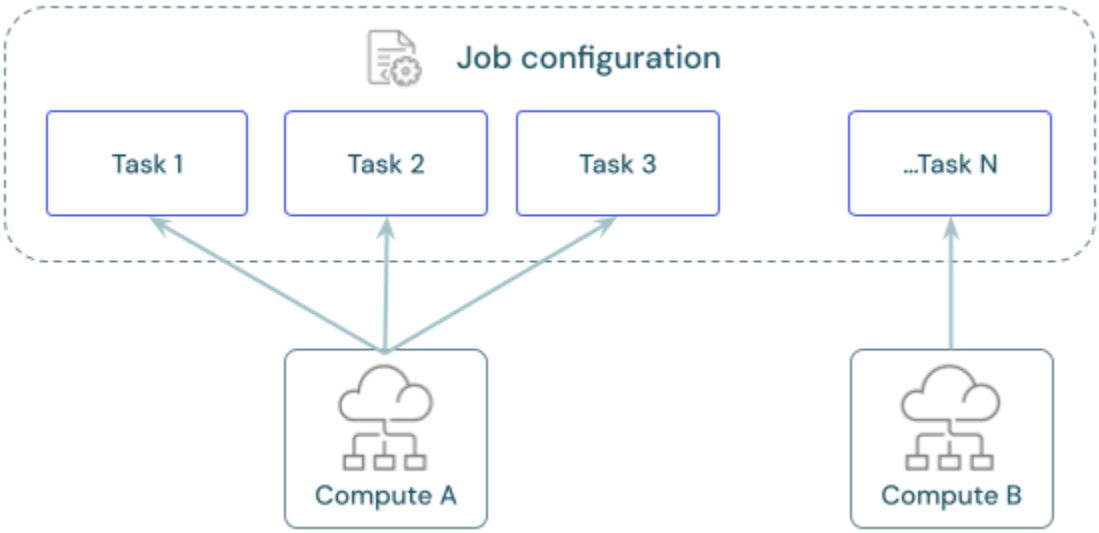
{{task.orders_silver_cleaned.values.has_dupicates}}

For accessing output from upstream task.

Compute Options, Workload Patterns, and Task Configuration:

Job can be executed on different compute:

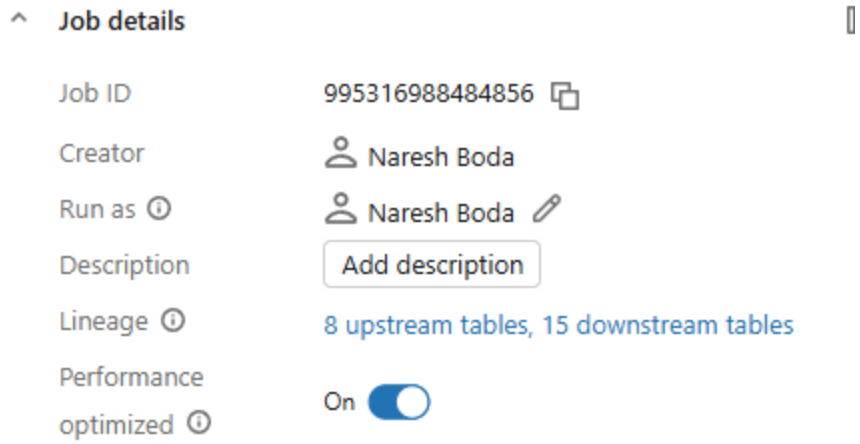
- interactive cluster
- job cluster
- serverless
- SQL warehouse



Each task within a job can have its own compute:

Task in the same job either:

- Share the same compute
- Use different computes as required.



Optimized ON is only applicable for serverless.

Performance optimized (OFF)

- Focus on cost efficiency
- Longer startup times (typically 4-6 minutes)
- Best for non urgent workloads

Performance optimized (ON)

- Faster job execution and startup
- Ideal for time sensitive workloads.
- Apply for serverless compute

Common workload patterns:

- **Sequential** (bronze → silver→ Gold)
- **Funnel** (multiple data sources, data collections)

Task 1

Task2 → task 4

Task3

- **Fan-out** (single data source, data distribution)

  Task2

  Task1

  Task3

  Task1 is distrubting task2 and task3

Common task configuration options:

- Task parameters
- Dynamic value reference
- Notification alert
- Retires policy

Retry policy:

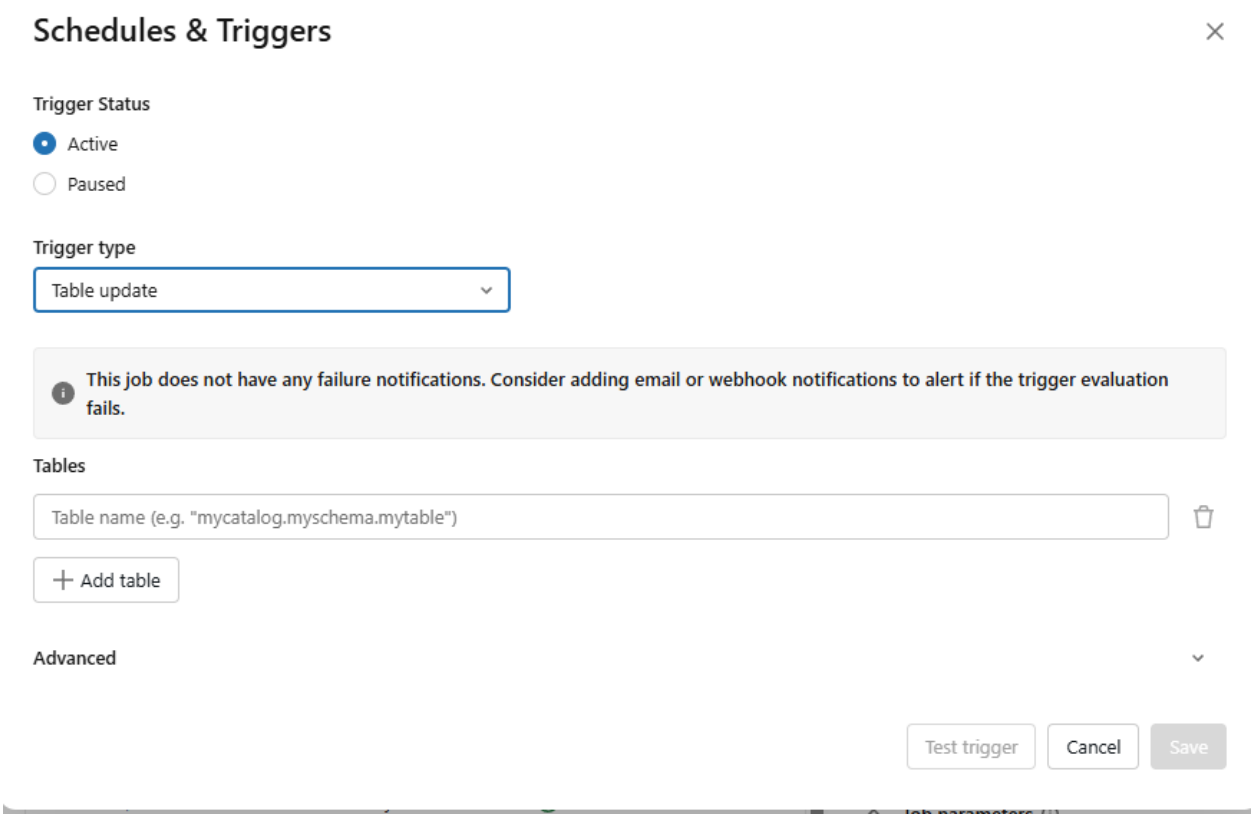
Consider factors like

- Failure types : transient network issues might require immediate retires, while data quality issues might not require immediate retry.
- Resource impact: retrying resource intensive tasks too aggressively can cause cluster resource contention.
- Downstream dependencies: failed tasks might impact other workflows, making retry interval critical.

***Triggers and For-Each Tasks*:**

A trigger is a rule that starts a job run based on specific condition or schedule.

1. Time based schedule
2. Continuous trigger (for streaming jobs – and always on)
3. File arrival event
4. Manual trigger
5. Table update



=> **Scheduled triggers** : cron expression for sophisticated schedule like “every 15 minutes during business hours” or “first Monday of each month”

Use case:

Daily ETL: process yesterday data every morning at 6AM.

Weekly reports : Monday morning.

=> **File arrival triggers** : automatically triggers job when new file are detected in a specified storage location.

Use case : partner data feed

=> **Continous triggrs** : runs a new job as soon as previous once finishes.

Use case:

- Fraud detection

=> manual triggers – lets jobs run on demand.

Best for adhoc runs, debugging, one off backfills.

- Backfill processing
- Emergency processing
- Testing and validation

=> table update triggers: automatically starts a job when specified tables are updated.

For each task:

- ***The*** *for each task – the top level container task that manages the loop*
- ***A nested*** *task – the actual task that executes each iteration*

Use case:

- Processing data for each state / region in parallel
- Time period processing : handles different data ranges with same logic.

Hard coded: ["US", "UK", "IN", "DE"]

Dynamic:

active_stores = ["store_001", "store_002", "store_003"]
dbutils.jobs.taskValues.set(key="store_list", value=active_stores)

Loop Configuration: In the For each task parameter box, you use a **Dynamic Value Reference** to capture that list:
{{tasks.get_active_stores.taskValues.store_list}}

Passing Data Down into the Nested Task:

[{"country": "US", "file_format": "parquet"}, {"country": "UK", "file_format": "csv"}]

You can extract specific variables using dot notation: {{input.country}} or {{input.file_format}}

Notebook reads the passed loop value

active_country = dbutils.widgets.get("country") df = spark.read.table(f"sales_{active_country}")

Best practice:

- *Use job or serverless cluster in production*
- *Use service principal for job ownership.*
- *Leverage the repair and run for reducing the cost of re processing.*
- *Paramterize tasks for reusablity and flexibility.*